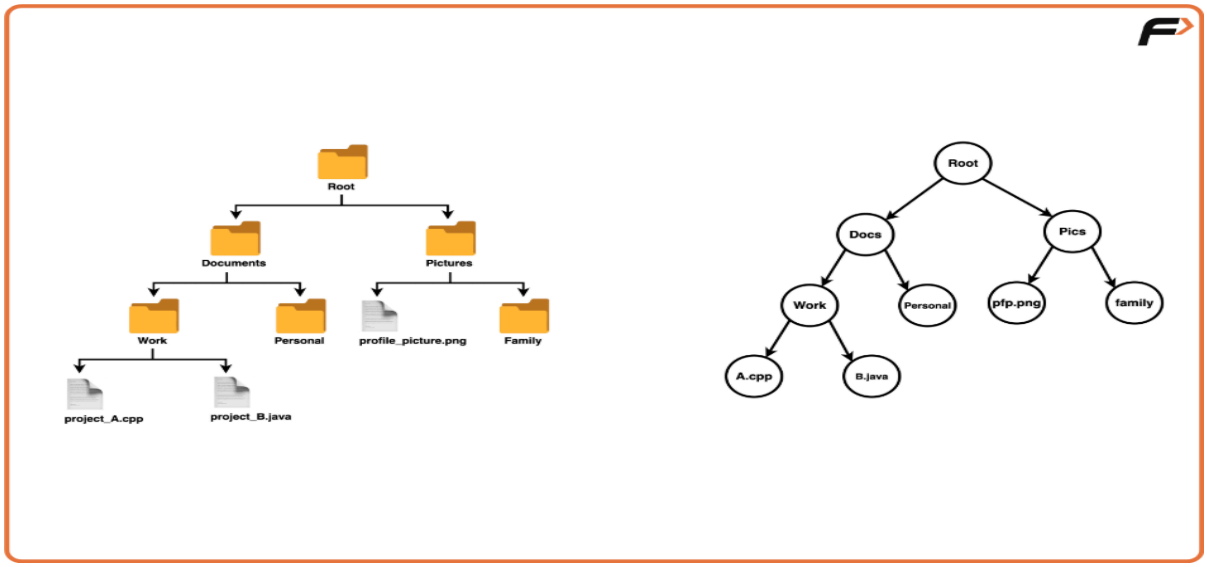
Binary Trees

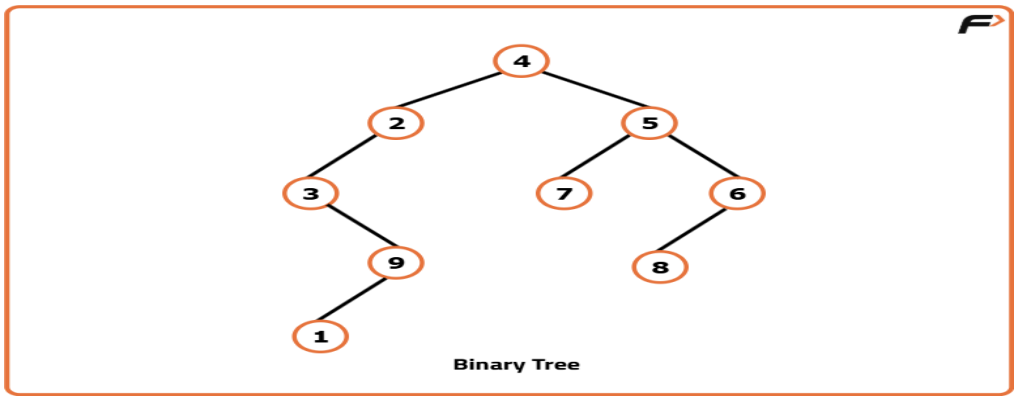
1. Introduction to Trees

In the world of data structures and algorithms, understanding binary trees lays the groundwork for hierarchical organisation and efficient data manipulation.

Up until now, we have studied array, linked list, stack and queues which are the fundamental linear data structures. Binary Trees are a different data structure and allow hierarchical organisation and structure of multi-level sequences. This resembles a tree with branching at each node expanding the tree in a non-linear fashion.

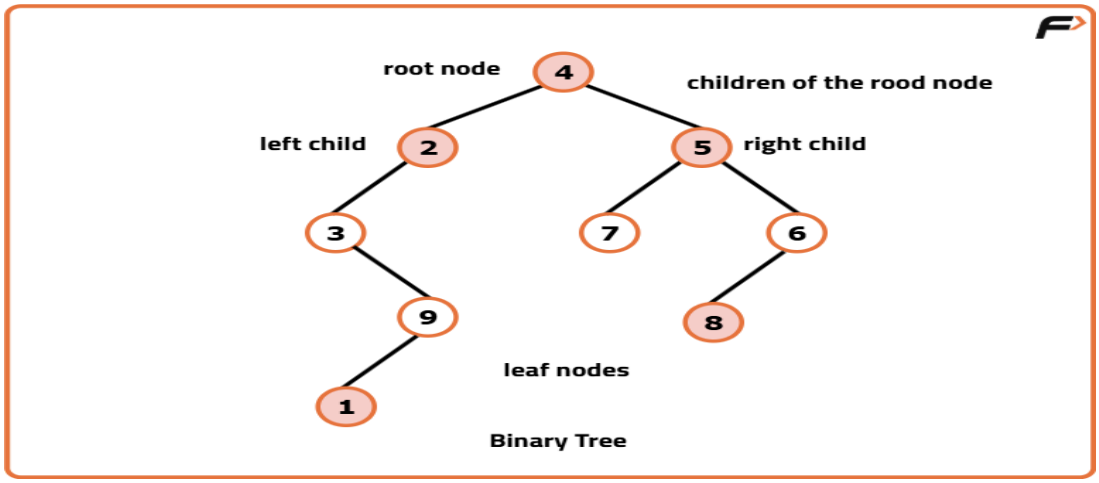


Just as the folders, subfolders and files are hierarchically arranged in your computer's file system, the binary tree has a similar structure with nodes representing folders and their children nodes representing the sub directory or files inside it.

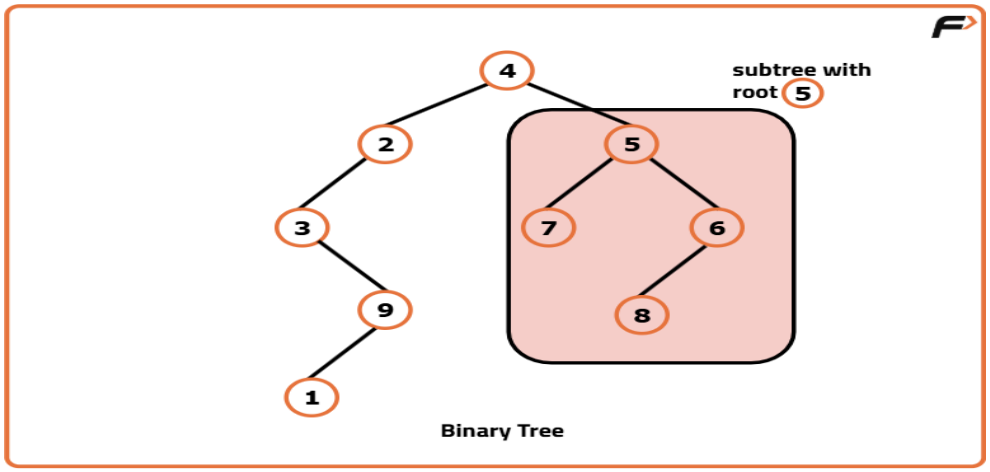


Binary Tree: Binary tree is a fundamental hierarchical data structure in computer science that comprises nodes arranged in a tree-like structure. It consists of nodes, where each node can have at most two children nodes, known as the left child and the right child.

Nodes: Each node in a binary tree contains a piece of data, often referred to as the node's value or key. This node also contains references and pointers to its left and right children so that we can traverse from one node to another in a hierarchical order.

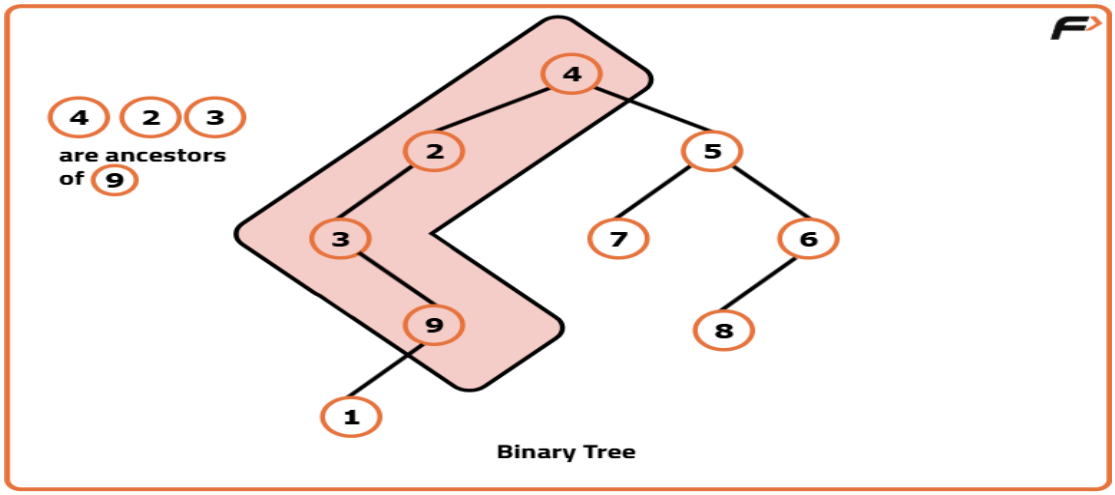


Root Node: is the topmost node of a binary tree from which all other nodes stem out. This serves as the entry point for traversing the tree structure.



Children Nodes are the nodes directly connected to a parent node. In a binary tree, a parent node can have zero, one or two children nodes, each situated to the left or right of the parent node.

Leaf Nodes are the nodes that do not have any children. These nodes lie on the outermost ends of the tree branches and are the terminal points of the traversal.

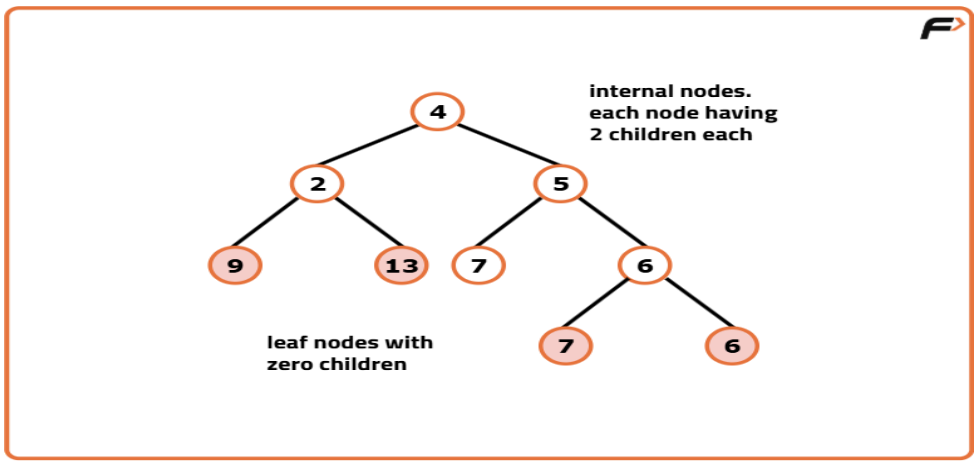


Ancestors in a Binary Tree are those nodes that lie on the path from a particular node to the root node. They are the nodes encountered while moving upwards from a specific node through its parent nodes until reaching the root of the tree.

Full Binary Tree:

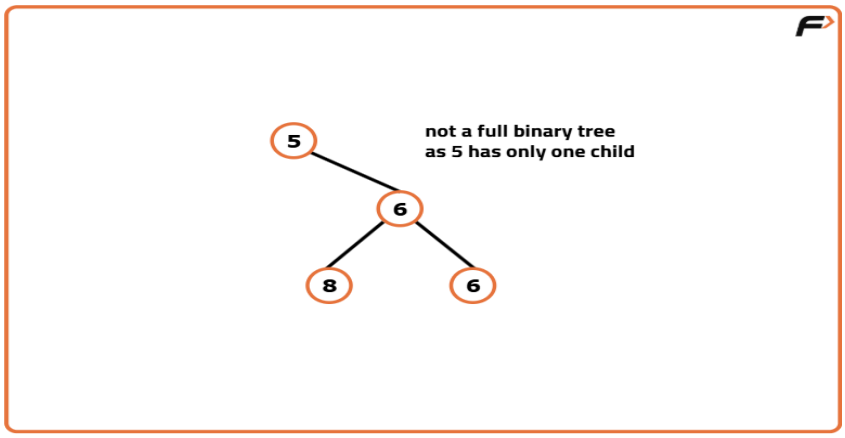
A Full Binary Tree, also known as a Strict Binary Tree, adheres to the structural property where every node has either zero or two children.

No node of this tree has just a single child, all internal nodes have exactly two children or in the case of leaf nodes, no children.



The property that each node has either 2 or 0 children contributes significantly to the tree's balance, making traversal, searching and insertion options more predictable and efficient.

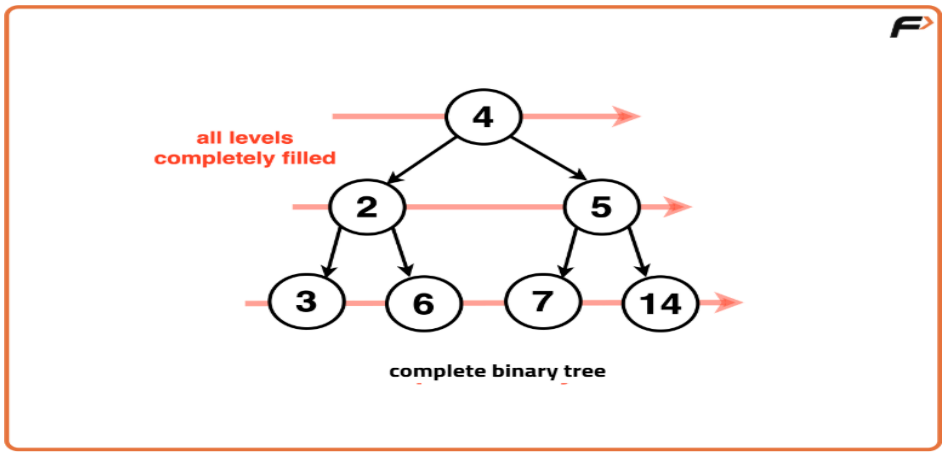
The emphasis that the internal nodes must have exactly two children optimises the tree's space utilisation and makes it more balanced.



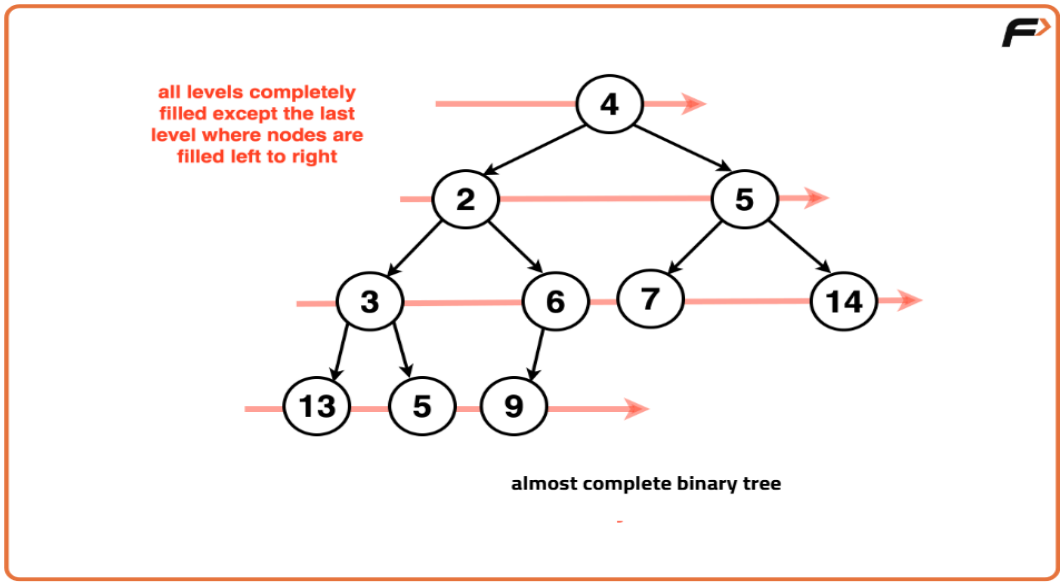
Complete Binary Tree:

A Complete Binary Tree is a specialised form of Binary Tree where all levels are filled completely except possibly the last level, which is filled from left to right.

All levels of the tree, except possibly the last one, are fully filled. If the last level is not completely filled, it is filled from left to right, ensuring that nodes are positioned as far left as possible.

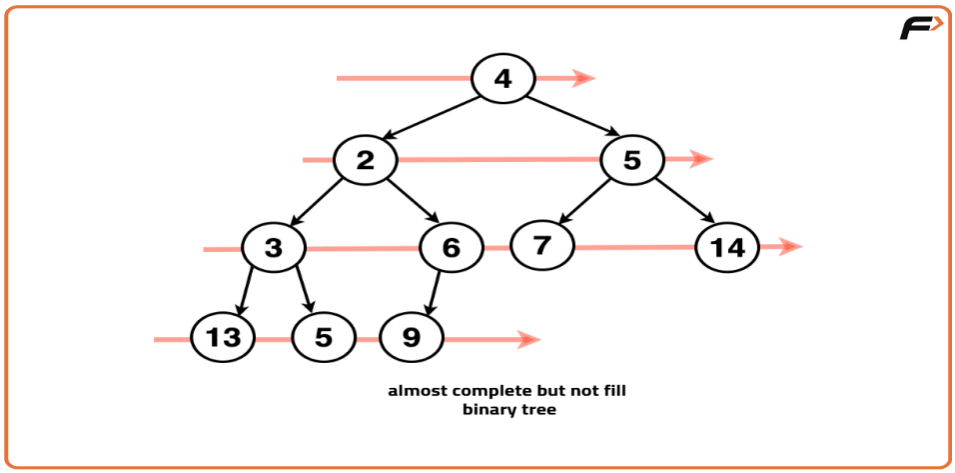


In a complete binary tree, all leaf nodes are in the last level or the second-to-last level, and they are positioned towards the leftmost side.



This structure is particularly useful for storing data in structures like heaps, where efficient access to the top element (root) or certain properties (e.g., maximum or minimum values in a heap) is crucial. The completeness property of a complete binary tree aids in achieving balanced structures, making it easier to implement algorithms and ensuring reasonably consistent performance.

One important aspect to note is that although it might seem similar to a full binary tree, a complete binary tree doesn't require all nodes to have two children; it's about the positioning and arrangement of nodes.



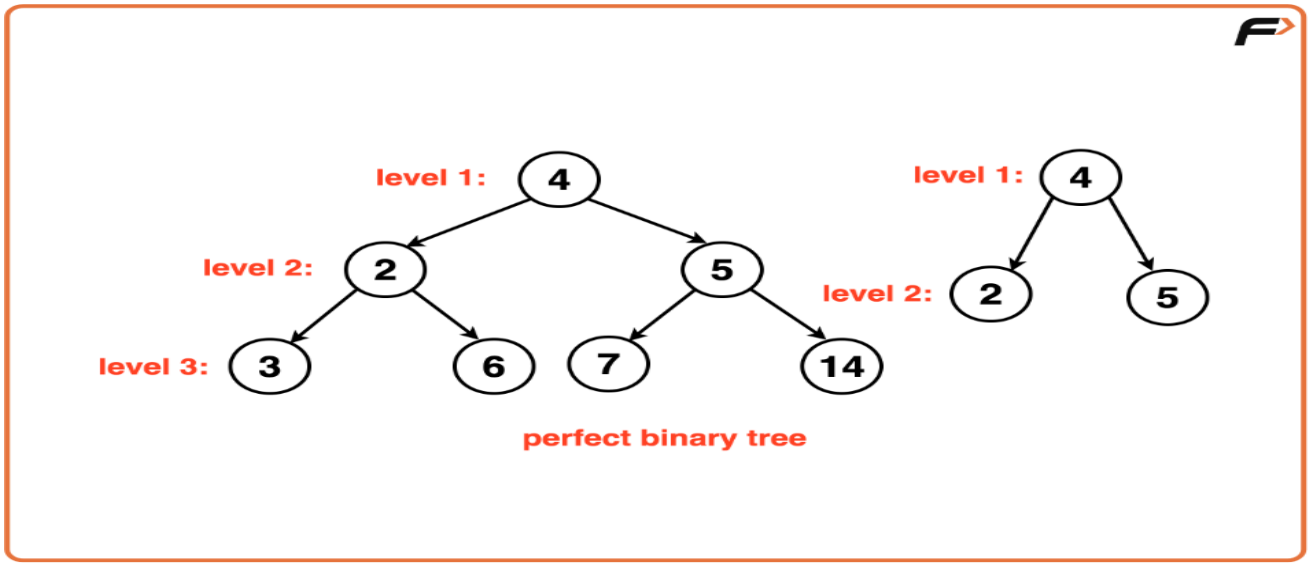
Perfect Binary Tree:

A Perfect Binary tree is a type of Binary Tree where all leaf nodes are at the same level and the number of leaf nodes is maximised for that level.

Every node in a perfect binary tree has either zero or two children. This means that every internal node (non-leaf node) has exactly two children and all leaf nodes are at the same level.

All levels of this tree are fully filled with nodes including the last level. Perfect Binary Trees have a balanced structure that maximises the number of nodes for a given height, creating a dense structure where the number of nodes doubles as we move down each level of the tree.

Properties of perfect binary trees make them efficient for certain operations like searching and sorting due to their balanced nature. However, achieving and maintaining perfect balance, especially when the number of nodes is not a power of two, might not be feasible in many practical applications.

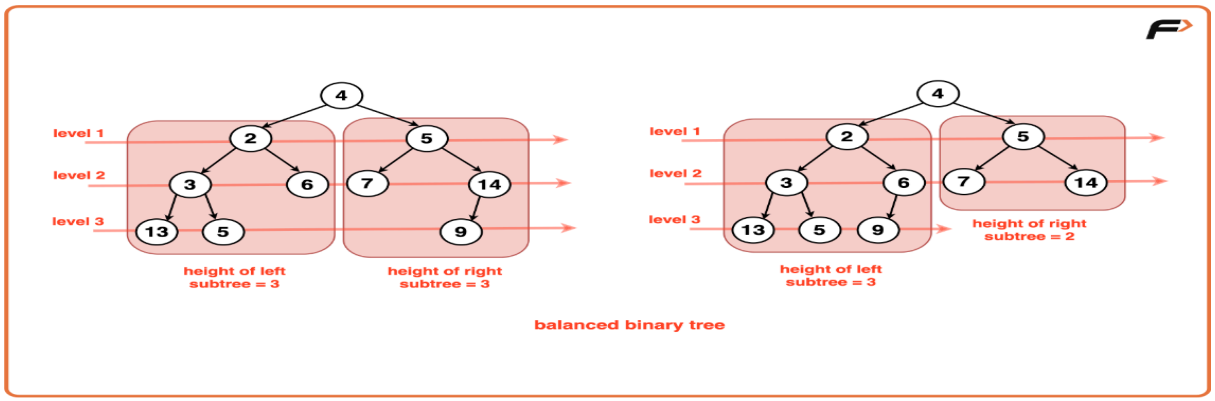


All levels of this tree are fully filled with nodes including the last level. Perfect Binary Trees have a balanced structure that maximises the number of nodes for a given height, creating a dense structure where the number of nodes doubles as we move down each level of the tree.

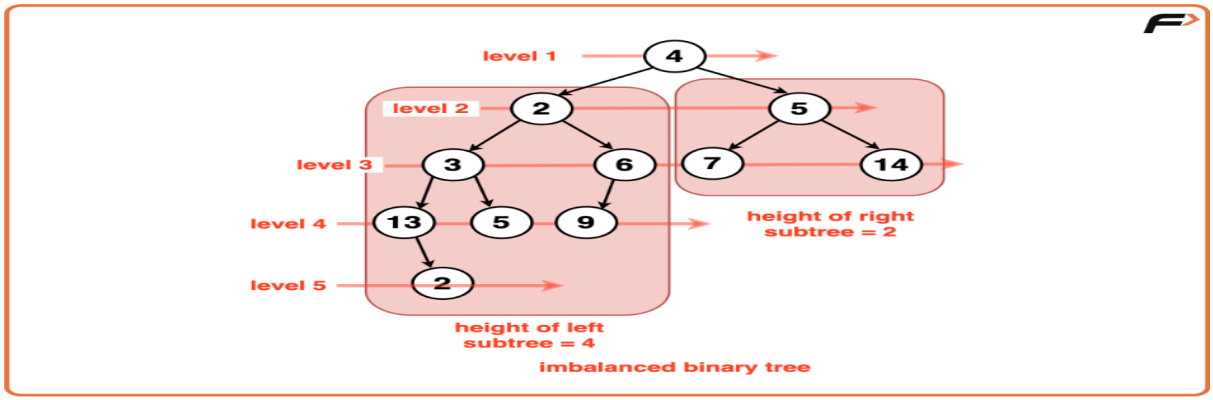
Properties of perfect binary trees make them efficient for certain operations like searching and sorting due to their balanced nature. However, achieving and maintaining perfect balance, especially when the number of nodes is not a power of two, might not be feasible in many practical applications.

Balanced Binary Tree:

A Balanced Binary tree is a type of Binary Tree where the heights of the two subtrees of any node differ by at most one. This property ensures that the tree remains relatively well-balanced, preventing the tree from becoming highly skewed or degenerate.

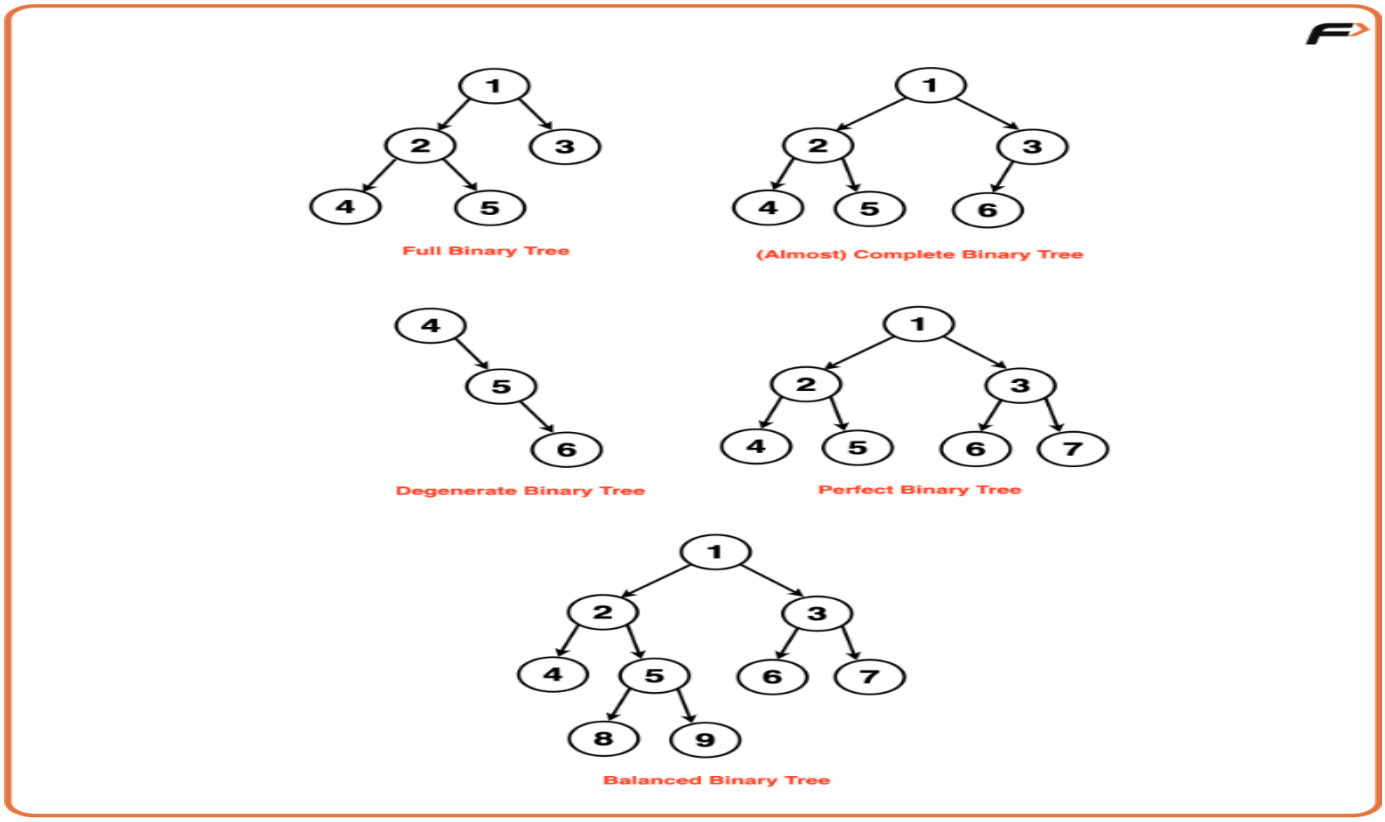


In a balanced binary tree, the height of the tree should be log2N at maximum, where N is the number of nodes. This ensures that the tree does not become heavily skewed or imbalanced. The distribution of nodes of both the left and right subtrees remains relatively even throughout the tree.



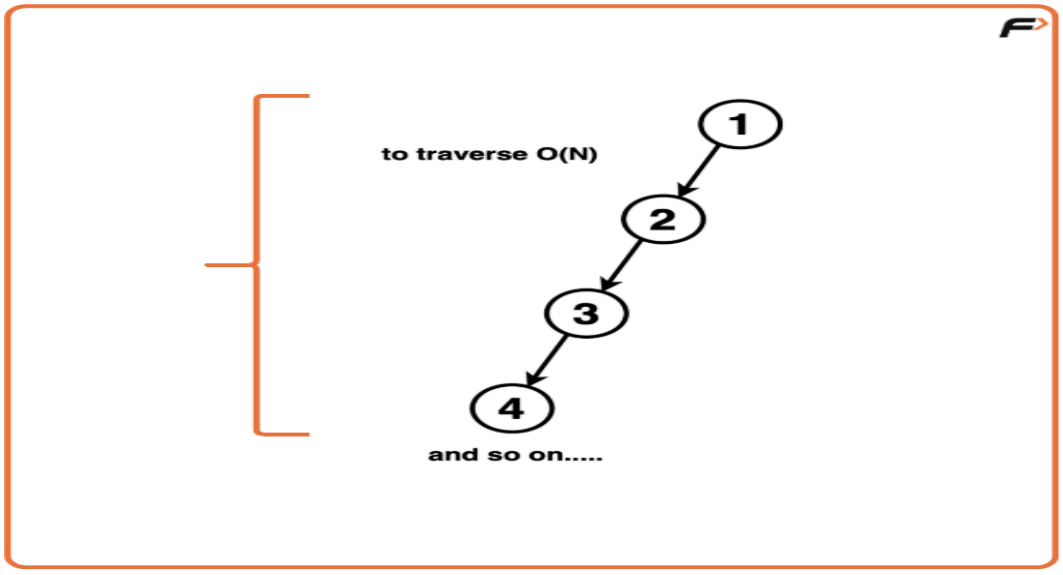
Degenerate Tree:

A Degenerate Tree is a Binary Tree where the nodes are arranged in a single path leaning to the right or left. The tree resembles a linked list in its structure where each node points to the next node in a linear fashion.



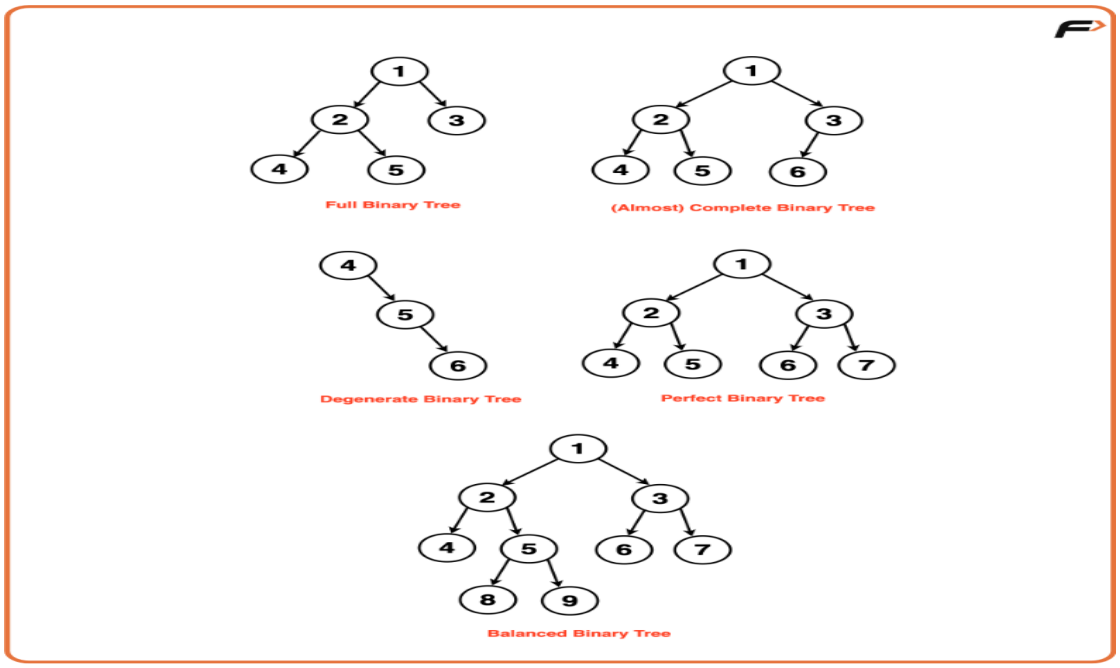
There's a lot of imbalance in the tree structure due to the sequential arrangement of nodes. This lack of balance results in a tree that resembles a singly linked list rather than a branching structure.

Each level of this tree only has one node. The height of tree reaches 'n' ie. the number of nodes in the tree, resulting in inefficient search operations. Though degenerate trees are not commonly used intentionally due to their inefficient nature for most operations, they may occur inadvertently in scenarios where nodes are inserted in a specific order (e.g., always to the right or left), causing the tree to lose its balanced properties.



Understanding degenerate trees is essential in analysing the worst-case time complexity of algorithms in scenarios where the tree structure degrades to this linear form, as it can help in optimising algorithms for these situations.

In Summary:



In summary, Binary Trees introduce a hierarchical arrangement taking a step ahead of the linear structure we have studied so far.

Binary Trees comprise of nodes, each capable of hosting at most two children hence the predecessor ‘Binary”. These structures mirror the hierarchical oraganisation seen in file systems.

Full Binary Trees impose the constraint that each node possesses either zero or two children, promoting a well-balanced structure and enhancing predictability in operations like traversal and insertion.

Complete Binary Trees, on the other hand, embrace a specialized form where all levels, save possibly the last, are completely filled. Their design, ensuring nodes are positioned leftmost on the last level, proves valuable for efficient data storage and access, resembling the organized arrangement of folders and files in a computer system.

Perfect Binary Trees take this hierarchical order a step further, showcasing a balanced structure where all leaf nodes align at the same level. Such trees optimize space by filling all levels with nodes, creating a dense structure.

Balanced Binary Trees ensure the difference in heights between subtrees of any node remains minimal, preventing significant skewing or imbalance.

Degenerate Trees represent a case where nodes arrange linearly, akin to a linked list, posing inefficiencies in search operations due to the lack of balance.

2. Binary Tree Representation in C++

A binary tree is a hierarchical data structure where each node can have at most two children: a left child and a right child.
In C++, a binary tree is commonly represented using pointers. Each node stores its data and pointers to its left and right child nodes. These pointers connect nodes together and form the tree structure.

A node may point to:

- a left child
- a right child
- or none (leaf node)

Approach 1

Algorithm

We represent a binary tree using a `Node` class.
Each node contains:

- an integer value (`data`)
- a pointer to the left child
- a pointer to the right child

Steps:

- Define a `Node` class with data, left pointer, and right pointer.
- Use a constructor to initialize the node value and set child pointers to `NULL`.
- Create nodes dynamically using `new`.
- Connect nodes by assigning the left and right pointers of a parent node.

Example explanation:

- Create a root node with value 1.
- Assign its left child as a node with value 2.
- Assign its right child as a node with value 3.
- Assign a left child to node 3 with value 5.

This forms the binary tree structure using pointers.

Code

```
#include <iostream>
```

```
using namespace std;

// Class representing a Node in the Binary Tree
class Node {
public:
    int data;      // Value stored in the node
    Node* left;    // Pointer to left child
    Node* right;   // Pointer to right child

    // Constructor
    Node(int val) {
        data = val;
        left = NULL;
        right = NULL;
    }
};

// Class to create a Binary Tree
class Solution {
public:
    Node* createBinaryTree() {
        // Create root node
        Node* root = new Node(1);

        // Create left and right children
        root->left = new Node(2);
        root->right = new Node(3);

        // Create further child
        root->right->left = new Node(5);

        return root;
    }
};

// Driver code
int main() {
    Solution sol;
```

```
    Node* root = sol.createBinaryTree();
    return 0;
}
```

Complexity Analysis
Time Complexity: O(N)
Creating N nodes takes linear time.

Space Complexity: O(N)
Memory is used to store N nodes of the binary tree.

3. Binary Tree Traversal : Inorder Preorder Postorder

Binary Tree traversal means visiting every node of the tree exactly once in a specific order.
Traversals are mainly divided into two types:

- Depth First Search (DFS): Goes deep into the tree before backtracking.
- Breadth First Search (BFS): Traverses the tree level by level.

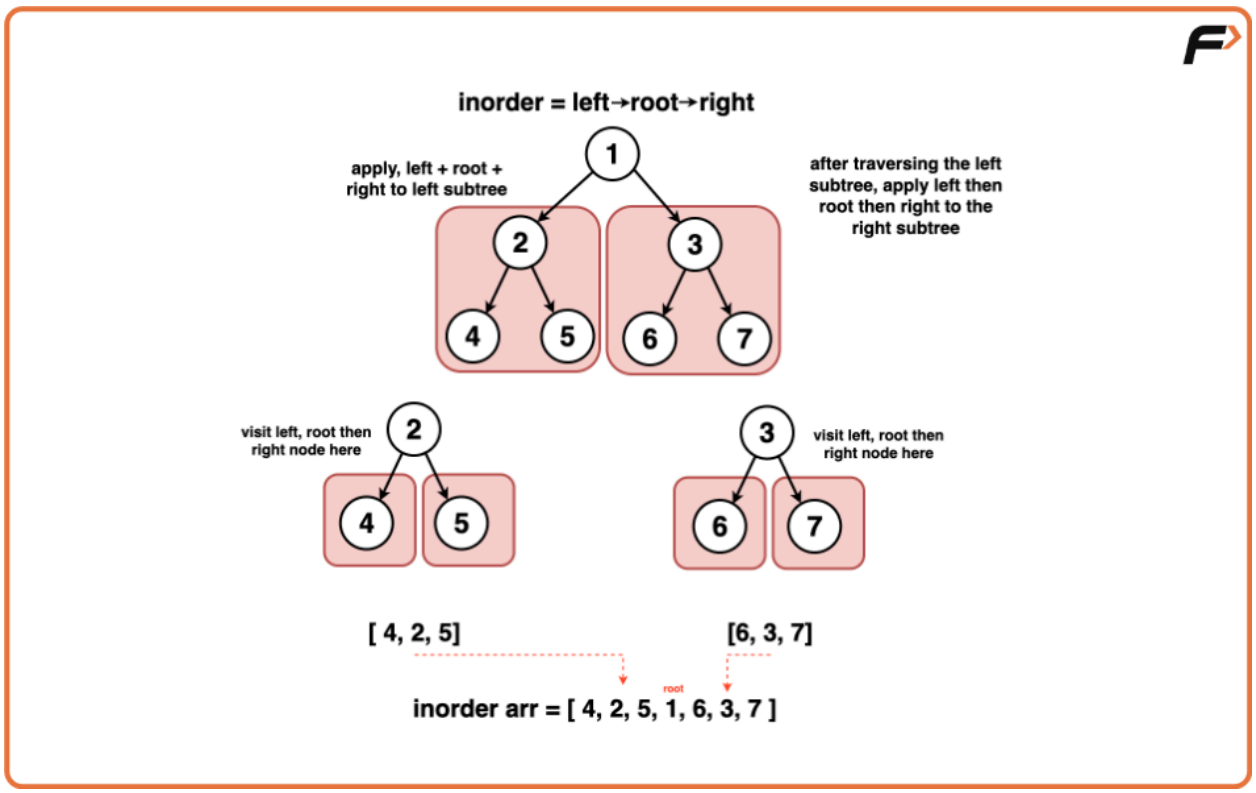
DFS includes Inorder, Preorder, and Postorder traversals.
BFS includes Level Order traversal.

Approach 1: Inorder Traversal (DFS)

In inorder traversal, nodes are visited in the order:
Left → Root → Right

This means we first visit the left subtree, then the current node, and finally the right subtree.

Example explanation:
For a tree with root 1, left child 2, and right child 3,
Inorder traversal will visit: 2 → 1 → 3



Algorithm

- If the current node is NULL, return.
- Recursively traverse the left subtree.
- Visit the current node (print or store its value).
- Recursively traverse the right subtree.

Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
class Node {
public:
    int data;
    Node* left;
    Node* right;
    Node(int val) {
        data = val;
        left = NULL;
        right = NULL;
    }
};

void inorder(Node* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);

    inorder(root);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)
Each node is visited once.

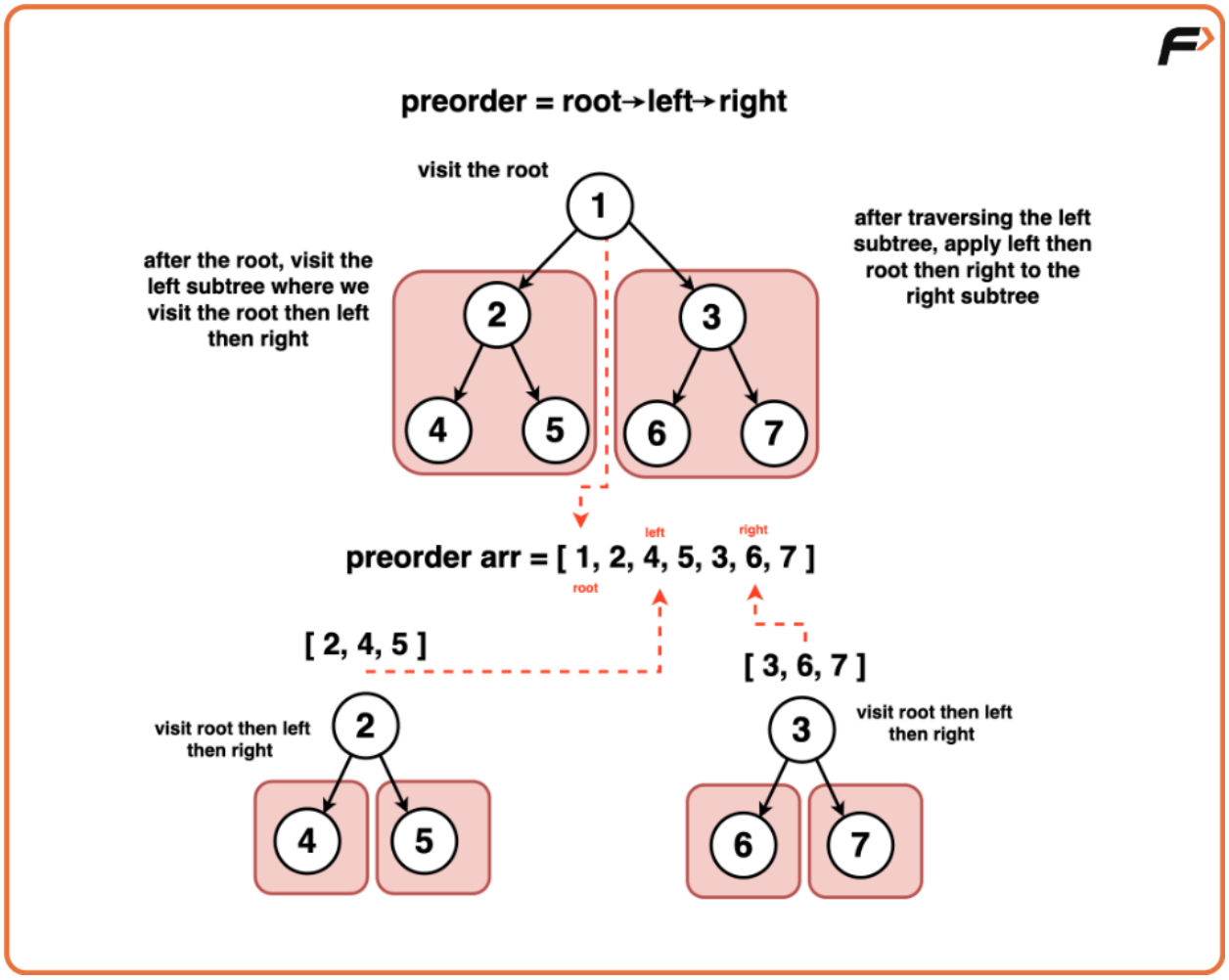
Space Complexity: O(N)
Recursive stack can go up to the height of the tree.

Approach 2: Preorder Traversal (DFS)

In preorder traversal, nodes are visited in the order:
Root → Left → Right

The current node is visited before its children.

Example explanation:
For the same tree, preorder traversal will visit: 1 → 2 → 3



Algorithm

- If the current node is NULL, return.
- Visit the current node.
- Recursively traverse the left subtree.
- Recursively traverse the right subtree.

Code

```
#include <bits/stdc++.h>
using namespace std;

void preorder(Node* root) {
    if (root == NULL) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}
```

Complexity Analysis
Time Complexity: O(N)
Each node is visited once.

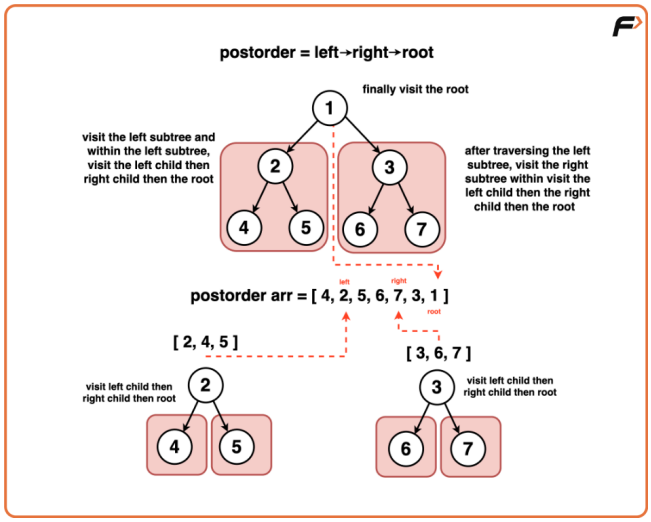
Space Complexity: O(N)
Space used by recursion stack.

Approach 3: Postorder Traversal (DFS)

In postorder traversal, nodes are visited in the order:
Left → Right → Root

The current node is visited after both children.

Example explanation:
For the same tree, postorder traversal will visit: 2 → 3 → 1



Algorithm

- If the current node is NULL, return.
- Recursively traverse the left subtree.
- Recursively traverse the right subtree.
- Visit the current node.

Code

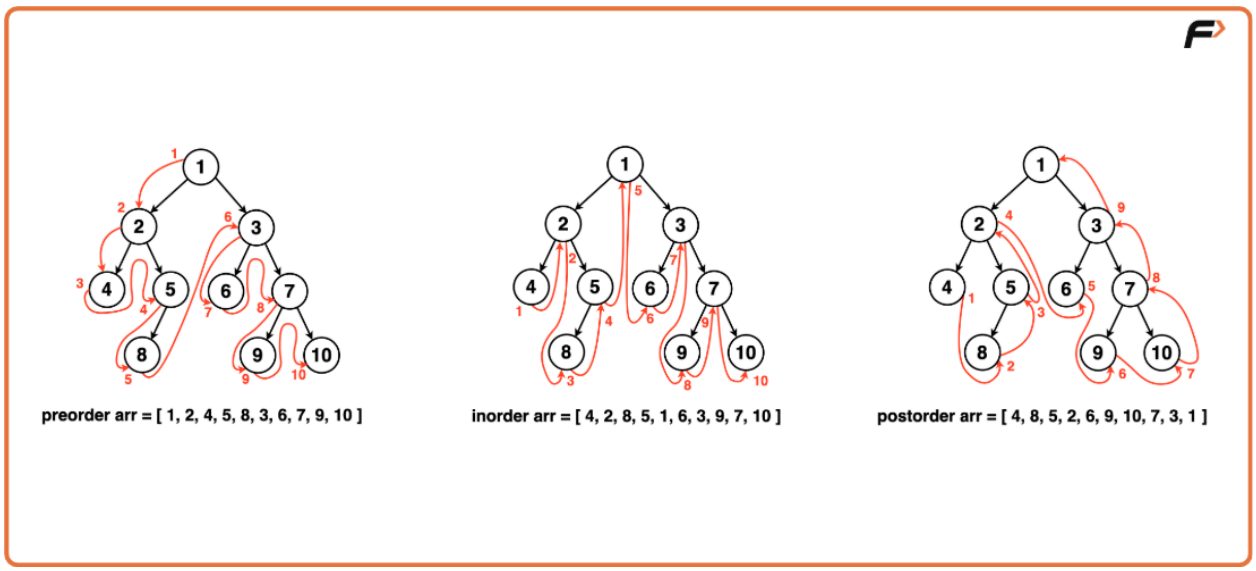
```
#include <bits/stdc++.h>
using namespace std;

void postorder(Node* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

Complexity Analysis

Time Complexity: O(N)
Each node is visited once.

Space Complexity: O(N)
Recursive stack space.



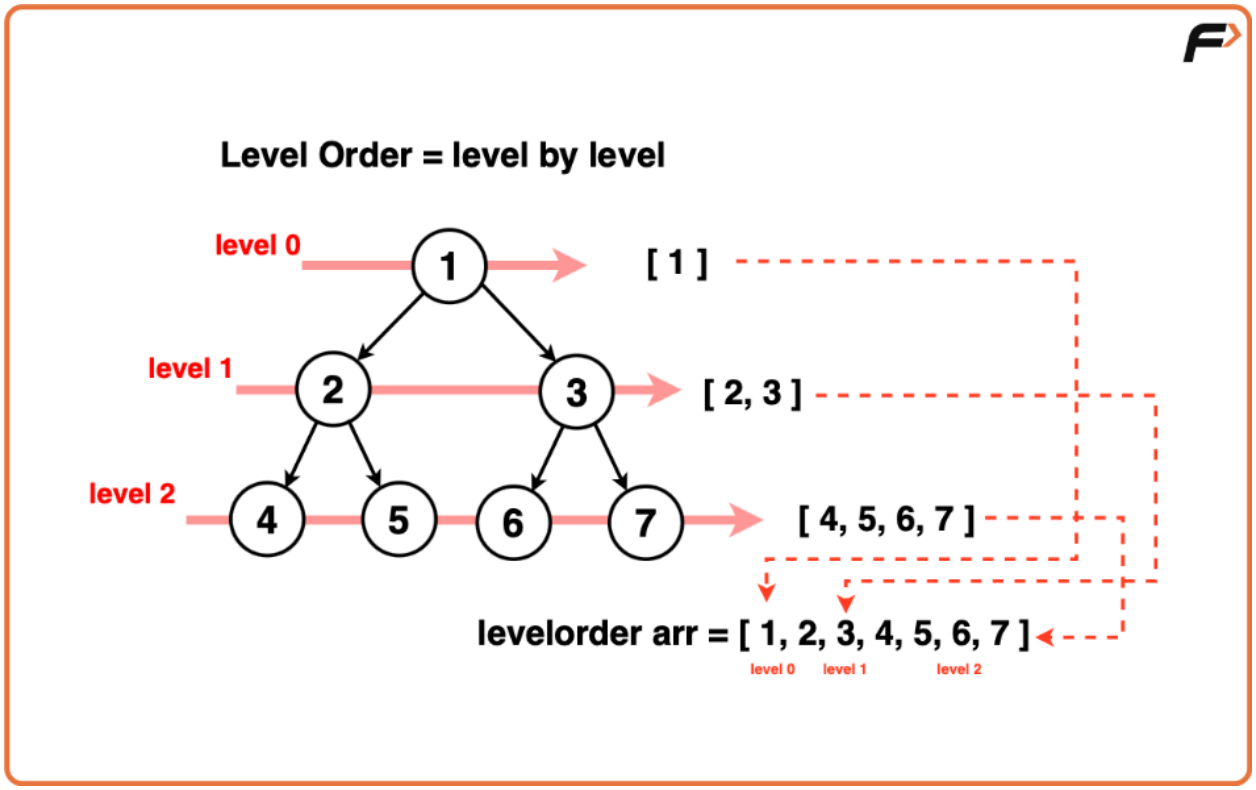
Approach 4: Level Order Traversal (BFS)

Level order traversal visits nodes level by level from left to right.

Example explanation:
For the tree, level order traversal will visit: 1 → 2 → 3

Algorithm

- If root is NULL, return.
- Create a queue and push the root node.
- While the queue is not empty:
    - Pop the front node and visit it.
    - Push its left child if it exists.
    - Push its right child if it exists.



Code

```
#include <bits/stdc++.h>
using namespace std;

void levelOrder(Node* root) {
    if (root == NULL) return;
    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* curr = q.front();
        q.pop();
        cout << curr->data << " ";

        if (curr->left) q.push(curr->left);
        if (curr->right) q.push(curr->right);
    }
}
```

Complexity Analysis
Time Complexity: O(N)
Each node is visited once.

Space Complexity: O(N)
Queue can store up to one full level of the tree.

4. Now Iterative

Approach 1: Inorder Traversal (Iterative)

In inorder traversal, we visit the left subtree first, then the root, and then the right subtree.
Using a stack, we keep moving left until we reach NULL, then process nodes while backtracking.

Example explanation:
For tree 1 2 3
Inorder result = 2 1 3

Algorithm

- Create an empty stack.
- Set current node as root.
- While current is not NULL or stack is not empty:
    - Push current node and move to left child.
    - When NULL is reached, pop from stack.
    - Visit the node.
    - Move to right child.

Code

```
#include <bits/stdc++.h>
```

```
using namespace std;

class Node {
public:
    int data;
    Node* left;
    Node* right;
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

void inorderIterative(Node* root) {
    stack<Node*> st;
    Node* curr = root;

    while (curr != NULL || !st.empty()) {
        while (curr != NULL) {
            st.push(curr);
            curr = curr->left;
        }
        curr = st.top();
        st.pop();
        cout << curr->data << " ";
        curr = curr->right;
    }
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);

    inorderIterative(root);
    return 0;
}
```

Complexity Analysis
Time Complexity: O(N)
Each node is pushed and popped once.

Space Complexity: O(N)
Stack space in worst case (skewed tree).

Approach 2: Preorder Traversal (Iterative)

In preorder traversal, we visit the root first, then left subtree, then right subtree.
We use a stack and always process the current node before pushing its children.

Example explanation:
For tree 1 2 3
Preorder result = 1 2 3

Algorithm

- If root is NULL, return.
- Push root into stack.
- While stack is not empty:
    - Pop top node and visit it.
    - Push right child first.
    - Push left child next.

Code

```
#include <bits/stdc++.h>
using namespace std;

void preorderIterative(Node* root) {
    if (root == NULL) return;

    stack<Node*> st;
    st.push(root);
```

```
    while (!st.empty()) {
        Node* curr = st.top();
        st.pop();
        cout << curr->data << " ";

        if (curr->right) st.push(curr->right);
        if (curr->left) st.push(curr->left);
    }
}
```

Complexity Analysis
Time Complexity: O(N)
Every node is visited once.

Space Complexity: O(N)
Stack space for nodes.

5. Level Order Traversal / Level Order Traversal in Spiral Form

You are given a Binary Tree.
Your task is to traverse the tree:

- Level Order Traversal (Normal BFS)
- Level Order Traversal in Spiral (Zig-Zag) Form

Level Order traversal visits nodes **level by level from left to right**.
Spiral Level Order traversal alternates direction at each level:

- Level 0 → left to right
- Level 1 → right to left
- Level 2 → left to right, and so on.

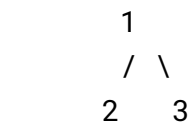
Approach 1: Level Order Traversal (BFS)

Algorithm
Level Order traversal uses a queue to process nodes level by level.

Steps:

- If root is NULL, return.
- Create a queue and push the root node.
- While the queue is not empty:
    - Take the front node from the queue.
    - Visit (print) the node.
    - Push its left child if it exists.
    - Push its right child if it exists.

Example explanation:
Tree:



Level order traversal → 1 2 3

Code

```
#include <bits/stdc++.h>
using namespace std;

class Node {
public:
    int data;
    Node* left;
    Node* right;
    Node(int val) {
```

```
        data = val;
        left = right = NULL;
    }
};

void levelOrder(Node* root) {
    if (root == NULL) return;

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* curr = q.front();
        q.pop();
        cout << curr->data << " ";

        if (curr->left) q.push(curr->left);
        if (curr->right) q.push(curr->right);
    }
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);

    levelOrder(root);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)
Each node is visited once.

Space Complexity: O(N)
Queue can store nodes of one complete level.

Approach 2: Level Order Traversal in Spiral Form (Zig-Zag)

Algorithm

Spiral traversal changes direction at every level.
We use a queue for level processing and a flag to control direction.

Steps:

- If root is NULL, return.
- Push root into a queue.
- Use a boolean flag `leftToRight` initially true.
- While queue is not empty:
    - Get the number of nodes at the current level.
    - Create a temporary array of that size.
    - For each node in the level:
        - Pop node from queue.
        - Place its value in correct position based on direction.
        - Push left and right children into the queue.
    - Print the temporary array.
    - Flip the direction flag.

Example explanation:
Tree:

```
    1
   / \
  2   3
```

Spiral order traversal → 1 3 2

Code

```
#include <bits/stdc++.h>
using namespace std;

void spiralLevelOrder(Node* root) {
    if (root == NULL) return;

    queue<Node*> q;
    q.push(root);
    bool leftToRight = true;

    while (!q.empty()) {
        int size = q.size();
        vector<int> level(size);

        for (int i = 0; i < size; i++) {
            Node* curr = q.front();
            q.pop();

            int index = leftToRight ? i : size - 1 - i;
            level[index] = curr->data;

            if (curr->left) q.push(curr->left);
            if (curr->right) q.push(curr->right);
        }

        for (int val : level)
            cout << val << " ";

        leftToRight = !leftToRight;
    }
}
```

Complexity Analysis

Time Complexity: O(N)
Each node is processed once.

Space Complexity: O(N)
Queue and temporary vector store nodes of one level.

6. Iterative Postorder Traversal of Binary Tree Using 2 Stacks

Given the root of a Binary Tree, we need to perform **postorder traversal** without using recursion.
Postorder traversal follows the order: **Left → Right → Root**.
The task is to return the traversal sequence using **two stacks**.

Example explanation (simple idea):
In recursive postorder, the root is processed after both children.
Using two stacks helps us reverse a modified preorder traversal to achieve postorder.

Approach 1: Using Two Stacks

Algorithm
We use two stacks to simulate postorder traversal.

Steps:

- If the root is NULL, return an empty list.
- Create two stacks: `st1` and `st2`.
- Push the root node into `st1`.
- While `st1` is not empty:
    - Pop a node from `st1`.
    - Push this node into `st2`.
    - Push the left child of the node into `st1` if it exists.
    - Push the right child of the node into `st1` if it exists.
- After the loop, `st2` contains nodes in reverse postorder.

- Pop all nodes from `st2` and store their values in the result array.
- Return the result array.

Dry run (simple tree):

```
   1
  / \
 2   3
```

- st1 process order: 1 → 2 → 3
- st2 stores: 1, 3, 2
- Final pop from st2 gives: 2 3 1 (postorder)

Postorder = reverse of (Root → Right → Left)

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

vector<int> postOrder(Node* root) {
    vector<int> postorder;
    if (root == NULL) return postorder;

    stack<Node*> st1, st2;
    st1.push(root);
```

```
    while (!st1.empty()) {
        Node* curr = st1.top();
        st1.pop();
        st2.push(curr);

        if (curr->left)
            st1.push(curr->left);
        if (curr->right)
            st1.push(curr->right);
    } //Root → Right → Left


    while (!st2.empty()) {
        postorder.push_back(st2.top()->data);
        st2.pop();
    } //Left → Right → Root


    return postorder;
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);

    vector<int> result = postOrder(root);
    for (int x : result) cout << x << " ";
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)

Each node is pushed and popped a constant number of times.

Space Complexity: O(N)
Two stacks together can store up to N nodes in the worst case.

7. Post-order Traversal of Binary Tree using 1 Stack

You are given the root of a Binary Tree.
Your task is to perform **postorder traversal** using **only one stack** (no recursion, no second stack) and return the traversal sequence.

Postorder traversal follows the order:
Left → Right → Root

This approach is more tricky than using two stacks because we must carefully track when a node's right subtree has already been processed.

Stack se recursion simulate karo aur `lastVisited` se yaad rakho

ki right subtree already process ho chuki hai ya nahi.

`lastVisited` ka role

Ye batata hai:

> "Kya main is node ke right subtree se wapas aa chuka hoon?"

Approach: Iterative Postorder Traversal Using One Stack

Algorithm
The idea is to simulate recursion using one stack and a pointer to track the last visited node.

Steps:

- Create an empty stack.
- Set `curr` to root and `lastVisited` to NULL.
- While `curr` is not NULL or stack is not empty:
    - If `curr` is not NULL:
        - Push `curr` into stack.
        - Move `curr` to its left child.
    - Else:
        - Peek the top node of stack.
        - If the right child exists and has not been visited yet:
            - Move `curr` to the right child.
        - Else:
            - Visit the node (add to result).
            - Mark it as last visited.
            - Pop it from stack.

This ensures that a node is processed only **after** both left and right subtrees are completed.

Example dry run (simple tree):

```
    1
   / \
  2   3
```

Traversal order will be: 2 → 3 → 1

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

vector<int> postOrder(Node* root) {
    vector<int> result;
    stack<Node*> st;
    Node* curr = root;
    Node* lastVisited = NULL;

    while (curr != NULL || !st.empty()) {
        if (curr != NULL) {
            st.push(curr);
            curr = curr->left;
```

```
        } else {
            Node* topNode = st.top();
            if (topNode->right != NULL && lastVisited !=
topNode->right) {
                curr = topNode->right;
            } else {
                result.push_back(topNode->data);
                lastVisited = topNode;
                st.pop();
            }
        }
    }
    return result;
}

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);

    vector<int> ans = postOrder(root);
```

```
    for (int x : ans) cout << x << " ";

    return 0;

}
```

Complexity Analysis
Time Complexity: O(N)
Each node is pushed and popped from the stack exactly once.

Space Complexity: O(N)
Stack can store up to N nodes in the worst case (skewed tree).

8. Preorder Inorder Postorder Traversals in One Traversal

You are given the root of a Binary Tree.
Your task is to return **preorder**, **inorder**, and **postorder** traversal sequences **by making only one traversal of the tree**.

Normally, each traversal requires a separate pass. Here, we do all three in a single traversal using a stack and a state mechanism.

Approach 1

Algorithm
We simulate recursion using a stack that stores a pair of:

- a tree node
- an integer state representing the traversal phase of that node

State meaning:

- 1 → Preorder state
- 2 → Inorder state
- 3 → Postorder state

Steps:

- If the tree is empty, return empty traversals.
- Create three arrays: `pre`, `in`, and `post`.
- Use a stack that stores `{node, state}`.
- Push the root node with state 1 onto the stack.
- While the stack is not empty:
    - Pop the top element.
    - If state is 1:
        - Add node value to preorder.
        - Change state to 2 and push it back.
        - Push left child with state 1 if it exists.
    - If state is 2:
        - Add node value to inorder.
        - Change state to 3 and push it back.
        - Push right child with state 1 if it exists.
    - If state is 3:
        - Add node value to postorder.
- Return all three traversal arrays.

Example dry run (simple tree):



Preorder → 1 2 3
Inorder → 2 1 3
Postorder → 2 3 1

All are produced in one traversal.

(node, state)

state	Meaning
1	node pe **first time** aaye
2	left subtree ho chuki
3	left + right dono ho chuke

🔹 State ka mapping

State	Action
1	Preorder → left jao

2 Inorder → right jao

3 Postorder → done

Code

```
#include <bits/stdc++.h>
using namespace std;

// Node structure
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) {
        data = val;
        left = right = NULL;
    }
};

class Solution {
public:
    vector<vector<int>> preInPostTraversal(Node* root) {
```

```
vector<int> pre, in, post;


if (root == NULL)
    return {};


stack<pair<Node*, int>> st;
st.push({root, 1});


while (!st.empty()) {
    auto it = st.top();
    st.pop();


    if (it.second == 1) {
        pre.push_back(it.first->data);
        it.second = 2;
        st.push(it);


        if (it.first->left)
            st.push({it.first->left, 1});
    }
    else if (it.second == 2) {
        in.push_back(it.first->data);
        it.second = 3;
```

```
                st.push(it);

                if (it.first->right)
                    st.push({it.first->right, 1});
            }
            else {
                post.push_back(it.first->data);
            }
        }

        return {pre, in, post};
    }
};

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);

    Solution sol;
    vector<vector<int>> res = sol.preInPostTraversal(root);
```

```
    cout << "Preorder: ";
    for (int x : res[0]) cout << x << " ";
    cout << endl;

    cout << "Inorder: ";
    for (int x : res[1]) cout << x << " ";
    cout << endl;

    cout << "Postorder: ";
    for (int x : res[2]) cout << x << " ";
    cout << endl;

    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)
Each node is processed a constant number of times.

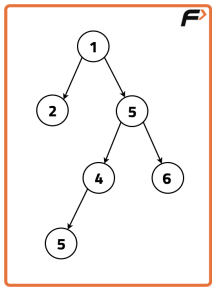
Space Complexity: O(N)
Space is used for the stack and for storing the three traversal arrays.

9. Maximum Depth of a Binary Tree

You are given the root of a Binary Tree.
Your task is to find the **height (maximum depth)** of the tree.

The height of a binary tree is defined as the **number of nodes on the longest path from the root node to any leaf node**.

Example:
Binary Tree: 1 2 5 -1 -1 4 6 5
The longest path is 1 → 5 → 4 → 5
So, the height of the tree is 4.



Approach 1

Algorithm
This approach uses **recursion** to calculate the height of the tree.

Steps:

- If the current node is `NULL`, return `0` (empty tree has height 0).
- Recursively calculate the height of the left subtree.
- Recursively calculate the height of the right subtree.
- The height of the current node is `1 + max(leftHeight, rightHeight)`.
- Return this value.

Example explanation:
For a tree with root 1:

- Height of left subtree = 3
- Height of right subtree = 1
- Height of tree = `1 + max(3, 1) = 4`

Code

```
#include <bits/stdc++.h>
using namespace std;

// Node structure for the binary tree
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    int maxDepth(Node* root) {
        if (root == NULL) {
            return 0;
        }

        int lh = maxDepth(root->left);
        int rh = maxDepth(root->right);

        return 1 + max(lh, rh);
    }
};

// Driver code
int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
```

```
    root->left->right = new Node(5);
    root->left->right->right = new Node(6);
    root->left->right->right->right = new Node(7);

    Solution solution;
    cout << solution.maxDepth(root);
    return 0;
}
```

Complexity Analysis
Time Complexity: O(N)
Each node of the binary tree is visited exactly once.

Space Complexity: O(N)
In the worst case (skewed tree), the recursion stack can go up to N nodes.

10. Check if the Binary Tree is Balanced Binary Tree

You are given a Binary Tree.
Your task is to check whether the tree is **balanced** or not.

A Binary Tree is said to be **balanced** if for **every node**, the absolute difference between the height of its left subtree and right subtree is **not more than 1**.

Example:
Input: `[3, 9, 20, null, null, 15, 7]`
Output: Yes
Because for every node, the height difference between left and right subtrees is at most 1.

Approach 1: Brute Force Approach

Algorithm

In this approach, we calculate the height of left and right subtrees for every node and check the balance condition.

Steps:

- If the root is NULL, return true (empty tree is balanced).
- Compute height of the left subtree.
- Compute height of the right subtree.
- Check if `abs(leftHeight - rightHeight) <= 1`.
- Recursively check if left subtree is balanced.
- Recursively check if right subtree is balanced.
- If all conditions are satisfied, return true; otherwise return false.

Example explanation:

At each node, height is calculated again and again for subtrees, which causes repeated work.

Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    bool isBalanced(Node* root) {
        if (root == NULL)
            return true;
```

```
        int leftHeight = getHeight(root->left);
        int rightHeight = getHeight(root->right);

        if (abs(leftHeight - rightHeight) <= 1 &&
            isBalanced(root->left) &&
            isBalanced(root->right))
            return true;

        return false;
    }

    int getHeight(Node* root) {
        if (root == NULL)
            return 0;

        int lh = getHeight(root->left);
        int rh = getHeight(root->right);
        return max(lh, rh) + 1;
    }
};

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->left->right->right = new Node(6);
    root->left->right->right->right = new Node(7);

    Solution sol;
    cout << sol.isBalanced(root);
    return 0;
}
```

Complexity Analysis

Time Complexity: $O(N^2)$

For every node, height calculation takes O(N) in the worst case.

Space Complexity: O(H)
Recursive call stack space, where H is the height of the tree.

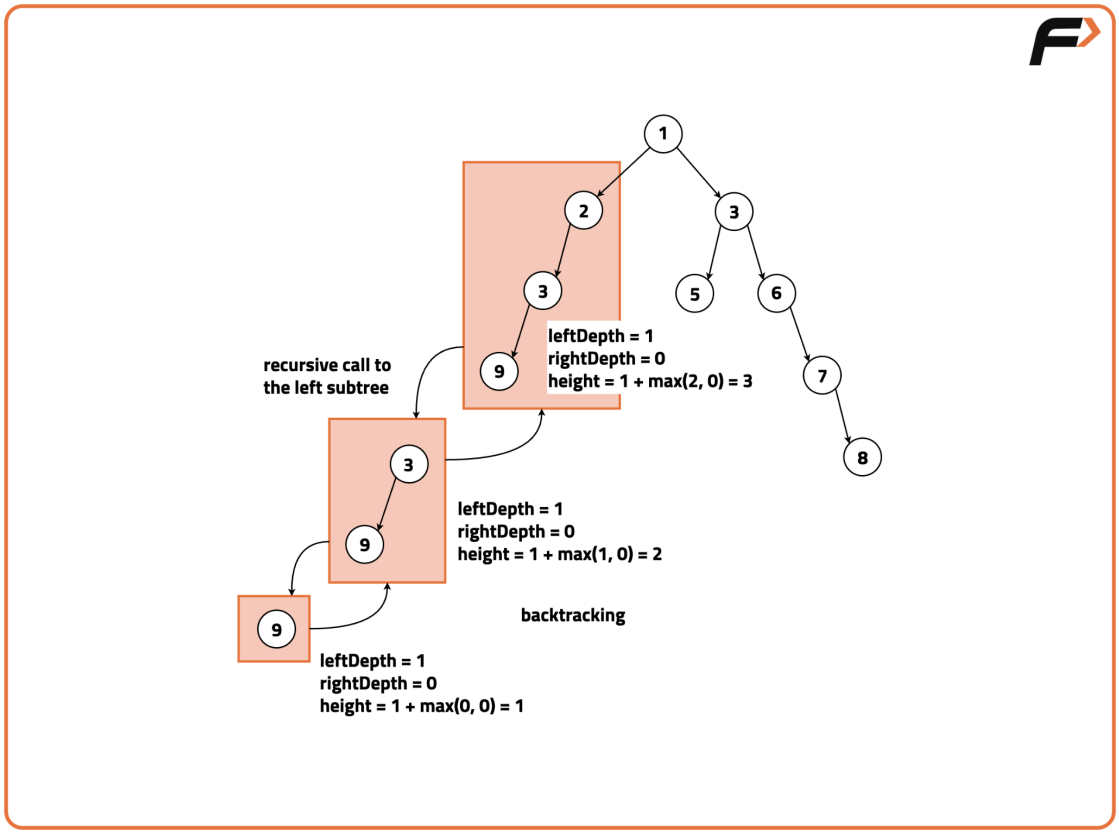
Approach 2: Optimal Approach

Algorithm
This approach avoids repeated height calculations by combining height computation and balance checking in a **single postorder traversal**.

Steps:

- Traverse the tree using postorder (Left → Right → Root).
- For each node:
    - Get height of left subtree.
    - Get height of right subtree.
- If any subtree is already unbalanced, return `-1`.
- If `abs(leftHeight - rightHeight) > 1`, return `-1`.
- Otherwise, return `max(leftHeight, rightHeight) + 1`.
- If final result is not `-1`, the tree is balanced.

Example explanation:
The moment an unbalanced node is found, we stop further computation and propagate `-1` upward.



Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    bool isBalanced(Node* root) {
        return dfsHeight(root) != -1;
    }

    int dfsHeight(Node* root) {
```

```
        if (root == NULL)
            return 0;

        int lh = dfsHeight(root->left);
        if (lh == -1)
            return -1;

        int rh = dfsHeight(root->right);
        if (rh == -1)
            return -1;

        if (abs(lh - rh) > 1)
            return -1;

        return max(lh, rh) + 1;
    }
};

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->left->right->right = new Node(6);
    root->left->right->right->right = new Node(7);

    Solution sol;
    cout << sol.isBalanced(root);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)

Each node is visited exactly once.

Space Complexity: O(H)

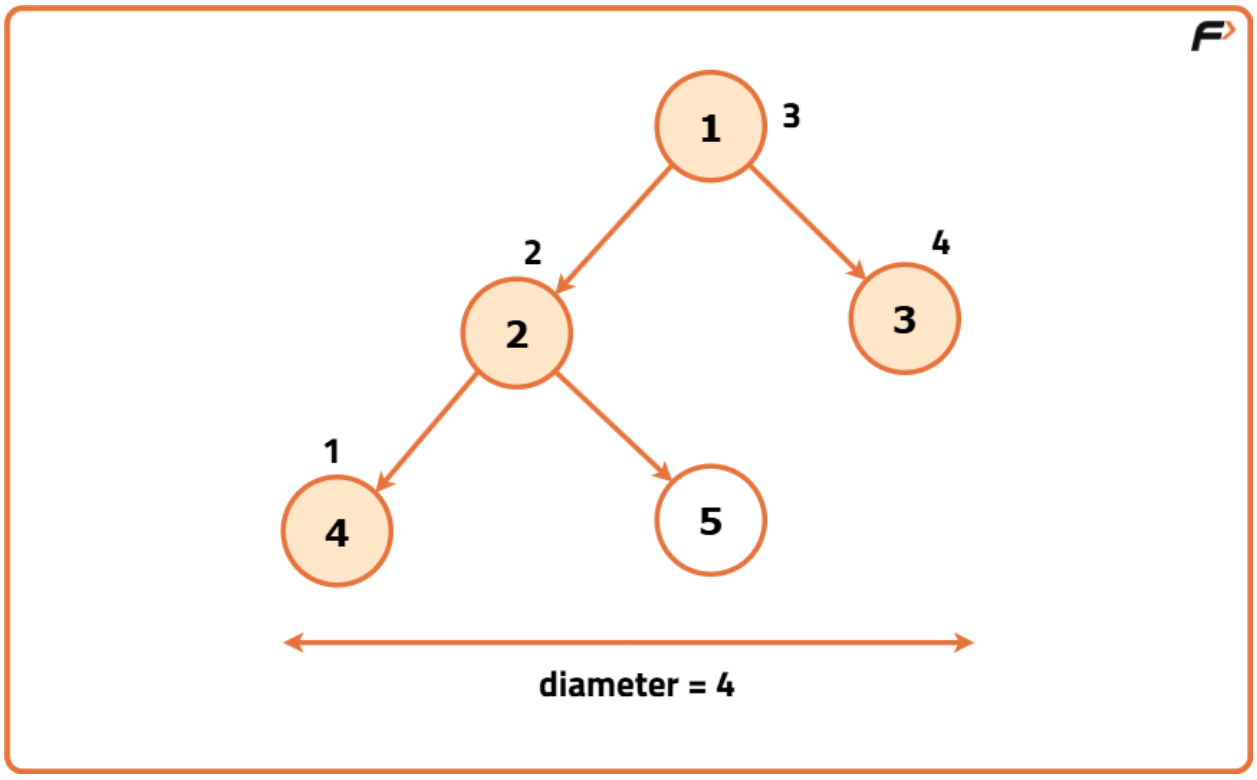
Only recursion stack space is used, where H is the height of the tree.

11. Calculate the Diameter of a Binary Tree

The diameter of a Binary Tree is defined as the **longest distance between any two nodes** in the tree.
This distance is measured as the **number of edges** on the longest path.
The path may or may not pass through the root.

Diameter = tree ke kisi bhi do nodes ke beech ka longest path (edges count)

⚠️ Path **root se jaana zaroori nahi**



Approach 1: Brute Force Approach

In this approach, we consider **every node as a turning point** of the diameter.

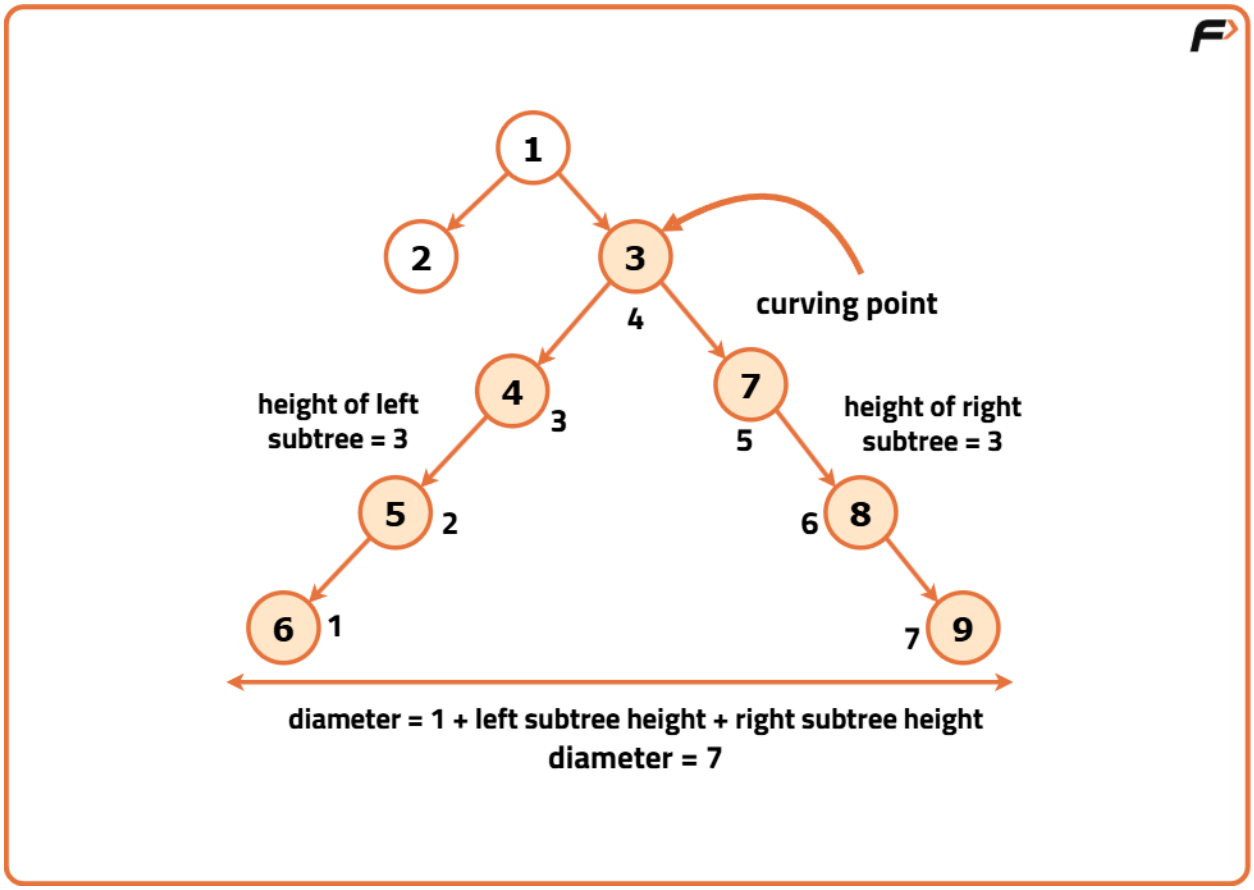
At each node:

- Find the height of the left subtree.
- Find the height of the right subtree.
- The diameter passing through that node is `leftHeight + rightHeight`.

We calculate this for all nodes and keep track of the maximum value.

Algorithm

- Initialize a variable `diameter` to store the maximum diameter.
- Define a recursive function to calculate height.
- For each node:
    - Recursively compute left subtree height.
    - Recursively compute right subtree height.
    - Update diameter as `max(diameter, leftHeight + rightHeight)`.
- Return the diameter.



Code

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    int diameter = 0;

    int height(Node* root) {
        if (root == NULL)
```

```
            return 0;

        int lh = height(root->left);
        int rh = height(root->right);

        diameter = max(diameter, lh + rh);

        return 1 + max(lh, rh);
    }

    int diameterOfBinaryTree(Node* root) {
        height(root);
        return diameter;
    }
};

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->left->right->right = new Node(6);
    root->left->right->right->right = new Node(7);

    Solution sol;
    cout << sol.diameterOfBinaryTree(root);
    return 0;
}
```

Complexity Analysis

- Time Complexity: **$O(N^2)$**
  Height is recomputed for each node.

- Space Complexity: **O(H)**
  Due to recursion stack, where H is the height of the tree.

Approach 2: Optimal Approach (Single Traversal)

The brute force approach is inefficient because heights are recalculated many times.
We can optimize this by calculating **height and diameter together** in a single postorder traversal.

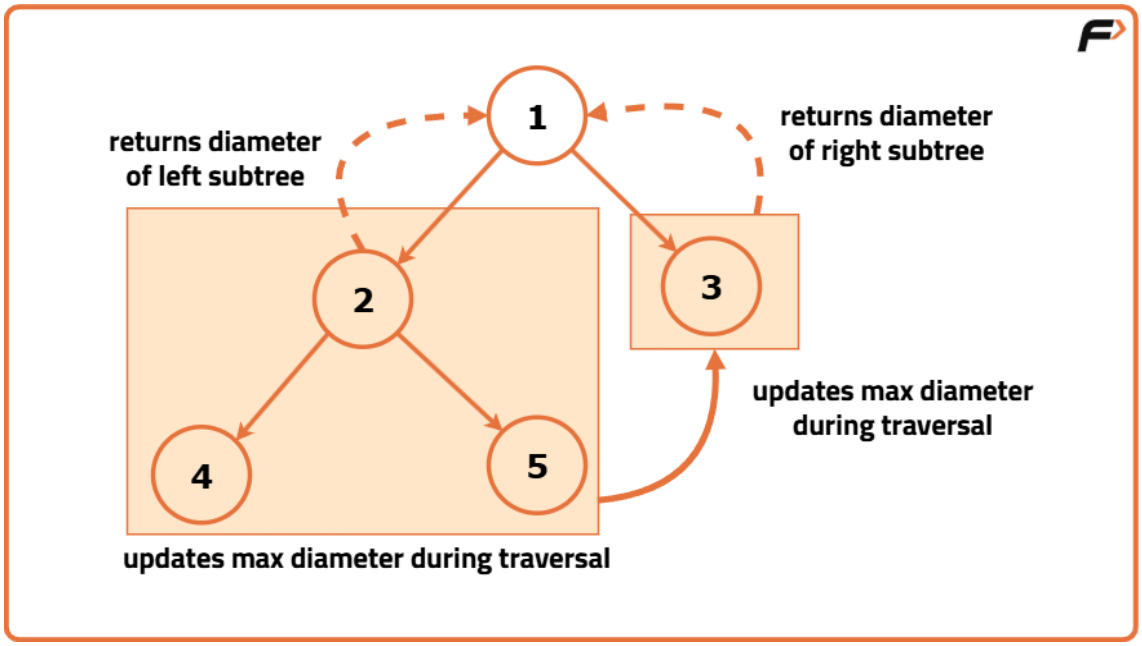
At each node:

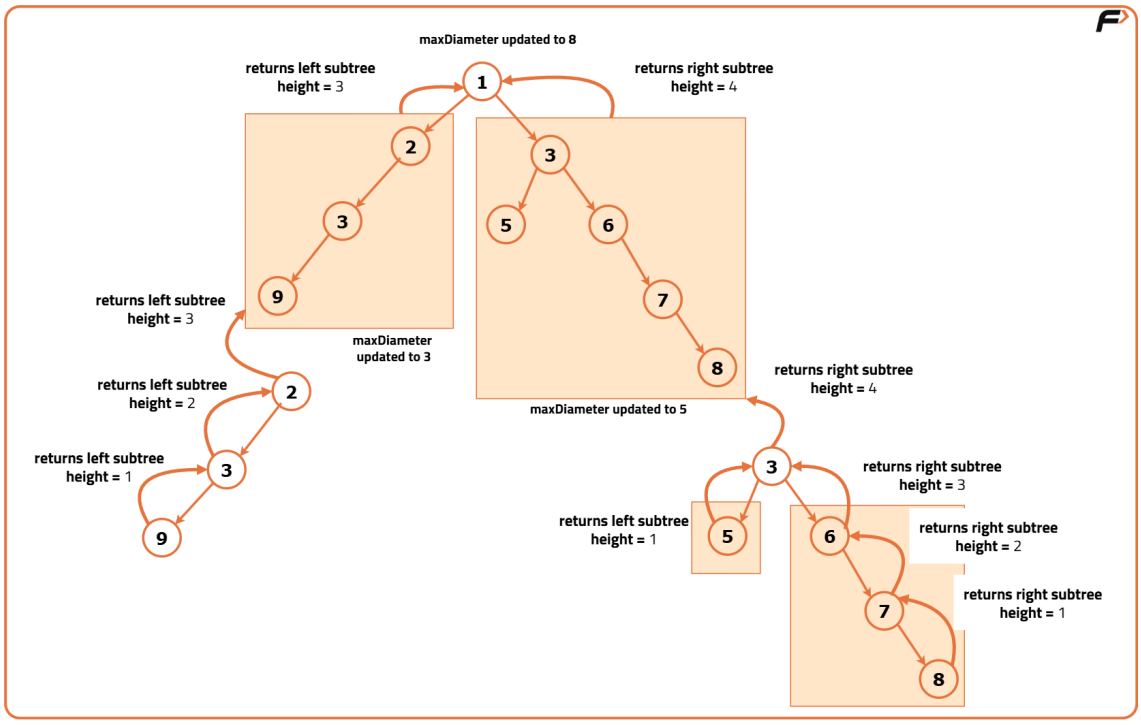
- Get left subtree height.
- Get right subtree height.
- Update diameter.
- Return the height to the parent.

This avoids repeated height calculations.

Algorithm

- Initialize diameter as 0.
- Use a recursive postorder function.
- For each node:
    - Compute left height.
    - Compute right height.
    - Update diameter as `max(diameter, leftHeight + rightHeight)`.
    - Return `1 + max(leftHeight, rightHeight)`.
- After traversal, return diameter.





Code

```
#include <bits/stdc++.h>
using namespace std;
```

```
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    int diameterOfBinaryTree(Node* root) {
        int diameter = 0;
        height(root, diameter);
        return diameter;
    }

    int height(Node* root, int& diameter) {
        if (root == NULL)
            return 0;

        int lh = height(root->left, diameter);
        int rh = height(root->right, diameter);

        diameter = max(diameter, lh + rh);

        return 1 + max(lh, rh);
    }
};

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->left->right->right = new Node(6);
    root->left->right->right->right = new Node(7);

    Solution sol;
```

```
    cout << sol.diameterOfBinaryTree(root);
    return 0;
}
```

Complexity Analysis

- Time Complexity: **O(N)**
  Each node is visited exactly once.

- Space Complexity: **O(H)**
  Recursion stack space, where H is the height of the tree.

12. Maximum Sum Path in Binary Tree

You are given a Binary Tree.
Your task is to find the **maximum sum possible along any path in the tree**.

A path:

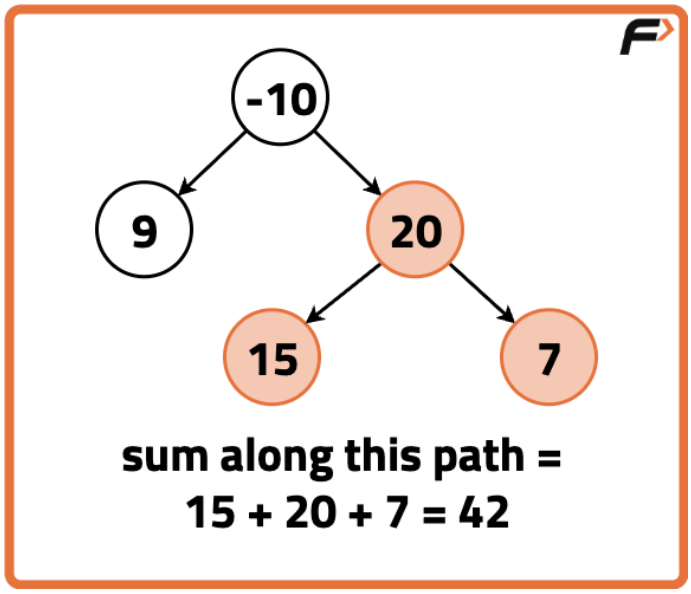
- Can start and end at **any node**.
- Must follow parent–child connections.
- Cannot visit the same node more than once.
- Does **not** need to pass through the root.

Question Explanation

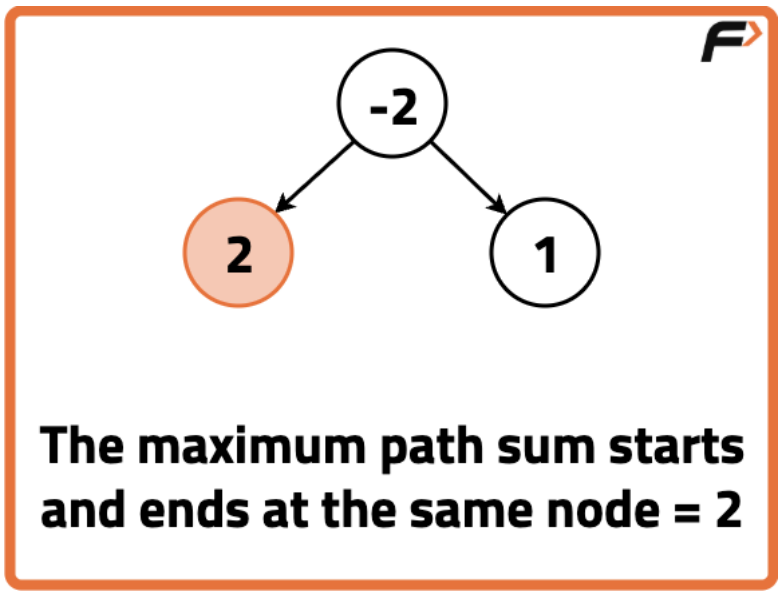
We need to find a path in the binary tree such that the **sum of node values on that path is maximum**.

Example 1:
Binary Tree: `-10 9 20 -1 -1 15 7`

The path 15 → 20 → 7 gives the maximum sum.
Sum = 15 + 20 + 7 = 42



Example 2:
Binary Tree: -2 2 1
The best path is just the node 2.
Sum = 2



Approach

Algorithm

We solve this using recursion and treat **every node as a possible turning point** of the path.

At each node, we calculate two things:

1. **Maximum path sum passing through the node**
   = node value + left subtree contribution + right subtree contribution
   This is used to update the global answer.

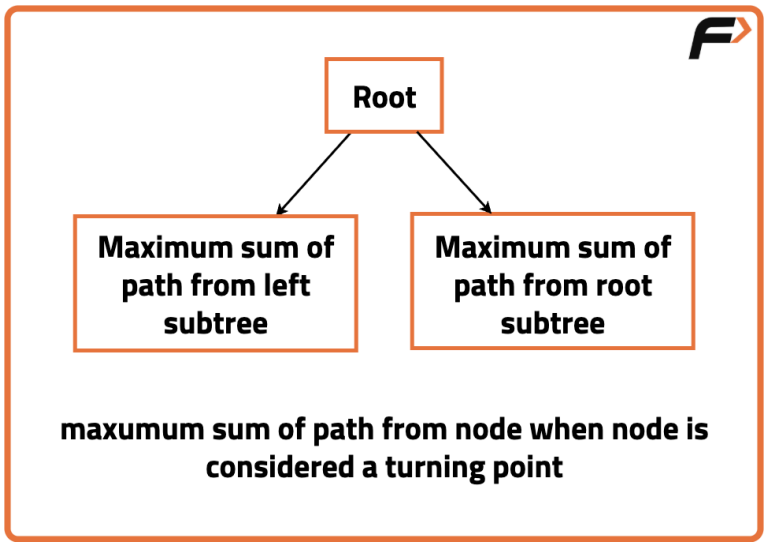
2. **Maximum one-sided path sum to return to parent**
   = node value + max(left contribution, right contribution)

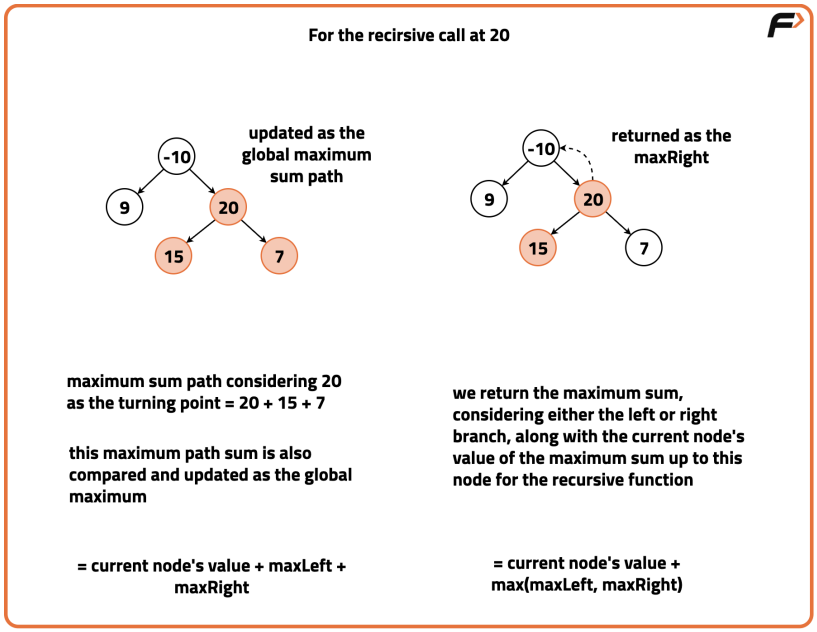
Important points:

- If a subtree gives a negative sum, we ignore it (use 0 instead).
- We keep a global variable to store the maximum path sum found so far.

Steps:

- If the node is NULL, return 0.
- Recursively compute left and right path sums.
- Ignore negative sums by taking `max(0, left/right)`.
- Update the global maximum using `left + right + node value`.
- Return the best one-sided path to the parent.





Code

```
#include <bits/stdc++.h>
using namespace std;

// Tree Node Definition
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
```

```
    TreeNode(int x) {
        val = x;
        left = NULL;
        right = NULL;
    }
};

class Solution {
public:
    int maxPathSum(TreeNode* root) {
        int maxSum = INT_MIN;
        dfs(root, maxSum);
        return maxSum;
    }

    int dfs(TreeNode* node, int &maxSum) {
        if (node == NULL)
            return 0;

        int left = max(0, dfs(node->left, maxSum));
        int right = max(0, dfs(node->right, maxSum));

        maxSum = max(maxSum, left + right + node->val);

        return max(left, right) + node->val;
    }
};

// Driver Code
int main() {
    TreeNode* root = new TreeNode(-10);
    root->left = new TreeNode(9);
    root->right = new TreeNode(20);
    root->right->left = new TreeNode(15);
    root->right->right = new TreeNode(7);

    Solution obj;
    cout << obj.maxPathSum(root);
```

```
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)
Each node is visited exactly once during DFS traversal.

Space Complexity: O(H)
Auxiliary space used by recursion stack, where H is the height of the binary tree.

13. Check if Two Trees are Identical

Given two Binary Trees, your task is to check whether they are **identical** or not.

Two binary trees are said to be identical if **all the following conditions are true for every corresponding node**:

- The values of the nodes are equal.
- The left subtrees are identical.
- The right subtrees are identical.

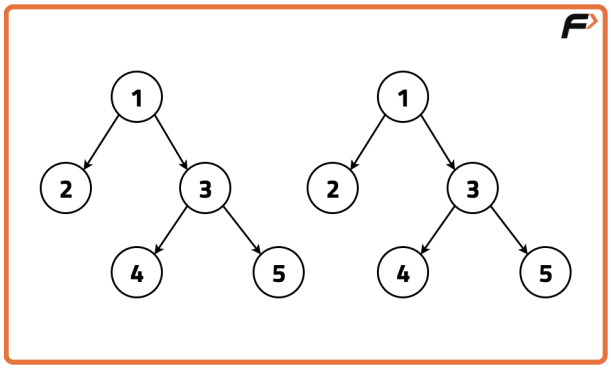
Question Explanation

We compare both trees **node by node**.

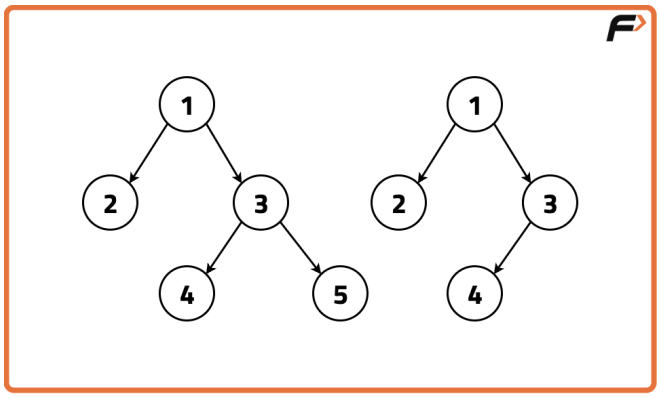
- If both nodes are `NULL`, they are identical up to this point.
- If one node is `NULL` and the other is not, the trees are not identical.
- If both nodes exist but their values are different, the trees are not identical.

- Otherwise, we recursively compare the left children and the right children.

Example 1:
Tree 1: 1 2 3 -1 -1 4 5
Tree 2: 1 2 3 -1 -1 4 5
Both structure and values match → **True**



Example 2:
Tree 1: 1 2 3 -1 -1 4 5
Tree 2: 1 2 3 -1 -1 4
Structure differs → **False**



Approach

Algorithm

- If both current nodes are NULL, return true.

- If only one node is NULL, return false.
- If values of current nodes are not equal, return false.
- Recursively check:
    - Left subtree of both trees
    - Right subtree of both trees
- If all checks pass, return true.

This ensures both **structure** and **data** are the same.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Node structure for the binary tree
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    bool isIdentical(Node* node1, Node* node2) {
        if (node1 == NULL && node2 == NULL)
            return true;

        if (node1 == NULL || node2 == NULL)
            return false;

        if (node1->data != node2->data)
            return false;
```

```
        return isIdentical(node1->left, node2->left) &&
               isIdentical(node1->right, node2->right);
    }
};

int main() {
    Node* root1 = new Node(1);
    root1->left = new Node(2);
    root1->right = new Node(3);
    root1->left->left = new Node(4);

    Node* root2 = new Node(1);
    root2->left = new Node(2);
    root2->right = new Node(3);
    root2->left->left = new Node(4);

    Solution sol;
    cout << sol.isIdentical(root1, root2);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N + M)
Each node of both trees is visited once, where `N` and `M` are the number of nodes in the two trees.

Space Complexity: O(H)
Only recursion stack space is used, where `H` is the height of the tree.

14. Zig Zag Traversal Of Binary Tree

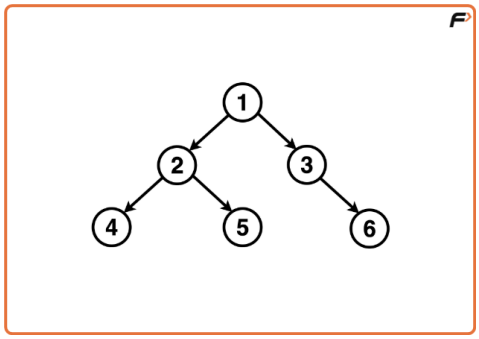
Given a Binary Tree, print its **zigzag (spiral) level order traversal**.
In zigzag traversal, the direction of traversal alternates at each level.

- Level 1: Left to Right
- Level 2: Right to Left
- Level 3: Left to Right
- and so on.

Question Explanation

We traverse the tree **level by level** like normal level order traversal, but the order of visiting nodes changes at each level.

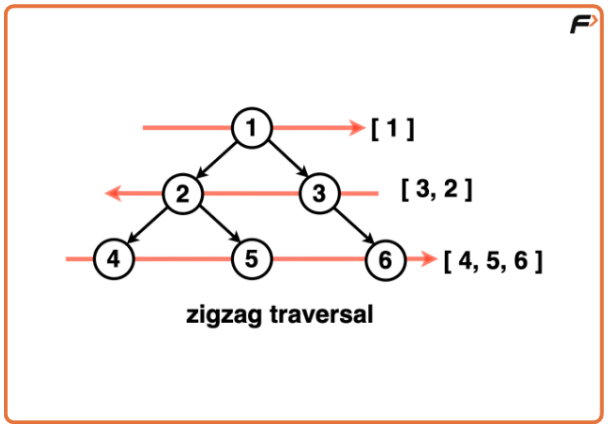
Example 1:
Binary Tree: `1 2 3 4 5 -1 6`



Traversal:

- Level 1 → `[1]` (left to right)
- Level 2 → `[3, 2]` (right to left)
- Level 3 → `[4, 5, 6]` (left to right)

Output: `[[1], [3, 2], [4, 5, 6]]`



Approach 1

Algorithm

This approach is based on **Level Order Traversal (BFS)** using a queue, with an extra flag to control direction.

Steps:

- If the root is NULL, return an empty result.
- Create a queue and push the root.
- Maintain a boolean flag `leftToRight`, initially set to true.
- While the queue is not empty:
    - Find the number of nodes at the current level.
    - Create a vector `level` of size equal to the number of nodes.
    - For each node at the current level:
        - Pop the node from the queue.
        - Decide the index in `level`:
            - If `leftToRight` is true, index = i.

        - Else, index = size - 1 - i.
      - Store the node value at the calculated index.
      - Push left and right children into the queue if they exist.
    - Add `level` to the answer.
    - Flip `leftToRight` for the next level.
- Return the final result.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Node structure for the binary tree
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        vector<vector<int>> ans;
        if (root == NULL) return ans;

        queue<TreeNode*> q;
        q.push(root);

        bool leftToRight = true;

        while (!q.empty()) {
            int size = q.size();
```

```
            vector<int> level(size);

            for (int i = 0; i < size; i++) {
                TreeNode* node = q.front();
                q.pop();

                int index = leftToRight ? i : size - 1 - i;
                level[index] = node->val;

                if (node->left) q.push(node->left);
                if (node->right) q.push(node->right);
            }

            ans.push_back(level);
            leftToRight = !leftToRight;
        }

        return ans;
    }
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->right->right = new TreeNode(6);

    Solution sol;
    vector<vector<int>> result = sol.zigzagLevelOrder(root);

    for (auto level : result) {
        cout << "[";
        for (int i = 0; i < level.size(); i++) {
            cout << level[i];
            if (i != level.size() - 1) cout << ", ";
        }
```

```
        cout << "] ";
    }
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)
Each node is visited exactly once during the level order traversal.

Space Complexity: O(N)
Queue can store up to O(N) nodes in the worst case, and the result vector also stores all nodes.

15. Boundary Traversal of a Binary Tree

Problem Statement
Given a Binary Tree, perform the **boundary traversal** of the tree in **anti-clockwise direction**, starting from the root.

Boundary traversal includes:

1. Root node
2. Left boundary (excluding leaf nodes)
3. All leaf nodes (from left to right)
4. Right boundary (excluding leaf nodes, added in reverse order)

Question Explanation

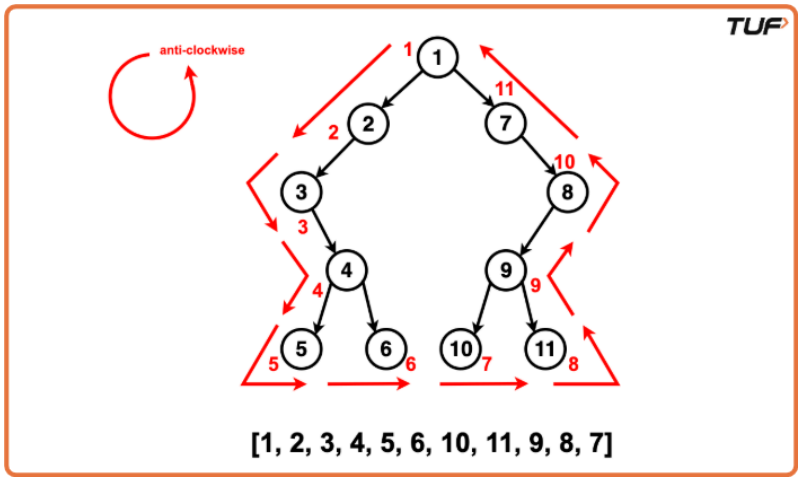
Boundary traversal means visiting only the **outer boundary nodes** of the tree.

Order of traversal (anti-clockwise):

- Start with the root.
- Traverse the left boundary top to bottom (excluding leaves).
- Traverse all leaf nodes from left to right.
- Traverse the right boundary bottom to top (excluding leaves).

Example 1
Binary Tree: `1 2 7 3 -1 -1 8 -1 4 9 -1 5 6 10 11`

Boundary Traversal:
`[1, 2, 3, 4, 5, 6, 10, 11, 9, 8, 7]`



Approach

Algorithm

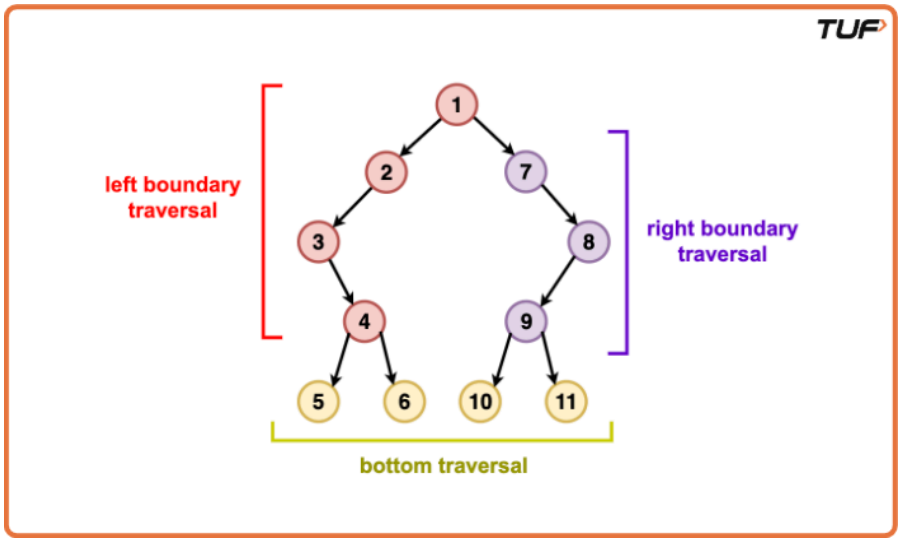
We break the boundary traversal into **three clear parts**.

1. **Left Boundary**
    - Start from root->left.

    - Keep moving to left child.
    - If left child does not exist, move to right child.
    - Add only non-leaf nodes.
2. **Leaf Nodes**
    - Traverse the whole tree using recursion.
    - Whenever a node is a leaf, add it to the result.
3. **Right Boundary**
    - Start from root->right.
    - Keep moving to right child.
    - If right child does not exist, move to left child.
    - Store non-leaf nodes in a temporary array.
    - Add them to result in reverse order.

Important:

- Leaf nodes must **not be repeated** in left or right boundary.
- Root should be added only once.



Code

```
#include <bits/stdc++.h>
using namespace std;

// Node structure
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    bool isLeaf(Node* node) {
        return node->left == NULL && node->right == NULL;
    }

    void addLeftBoundary(Node* root, vector<int>& res) {
        Node* curr = root->left;
        while (curr) {
            if (!isLeaf(curr))
                res.push_back(curr->data);
```

```
        if (curr->left)
            curr = curr->left;
        else
            curr = curr->right;
    }
}

void addLeaves(Node* root, vector<int>& res) {
    if (root == NULL) return;

    if (isLeaf(root)) {
        res.push_back(root->data);
        return;
    }
    addLeaves(root->left, res);
    addLeaves(root->right, res);
}

void addRightBoundary(Node* root, vector<int>& res) {
    Node* curr = root->right;
    vector<int> temp;
    while (curr) {
        if (!isLeaf(curr))
            temp.push_back(curr->data);
        if (curr->right)
            curr = curr->right;
        else
            curr = curr->left;
    }
    for (int i = temp.size() - 1; i >= 0; i--)
        res.push_back(temp[i]);
}

vector<int> boundaryTraversal(Node* root) {
    vector<int> res;
    if (root == NULL) return res;

    if (!isLeaf(root))
```

```
            res.push_back(root->data);

        addLeftBoundary(root, res);
        addLeaves(root, res);
        addRightBoundary(root, res);

        return res;
    }
};

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(5);
    root->right->left = new Node(6);
    root->right->right = new Node(7);

    Solution sol;
    vector<int> ans = sol.boundaryTraversal(root);

    for (int x : ans)
        cout << x << " ";
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)
Every node is visited at most once while finding boundaries and leaves.

Space Complexity: O(N)
Result vector stores boundary nodes.
Recursive stack space can go up to O(H), where H is height of tree.

16. Vertical Order Traversal of Binary Tree

Problem Statement
Given a Binary Tree, return its **Vertical Order Traversal** from the **leftmost vertical level to the rightmost vertical level**.
If multiple nodes lie on the same vertical line, they must be printed in **level order** (top to bottom, left to right).

Question Explanation

In vertical order traversal, we imagine vertical lines passing through the tree.

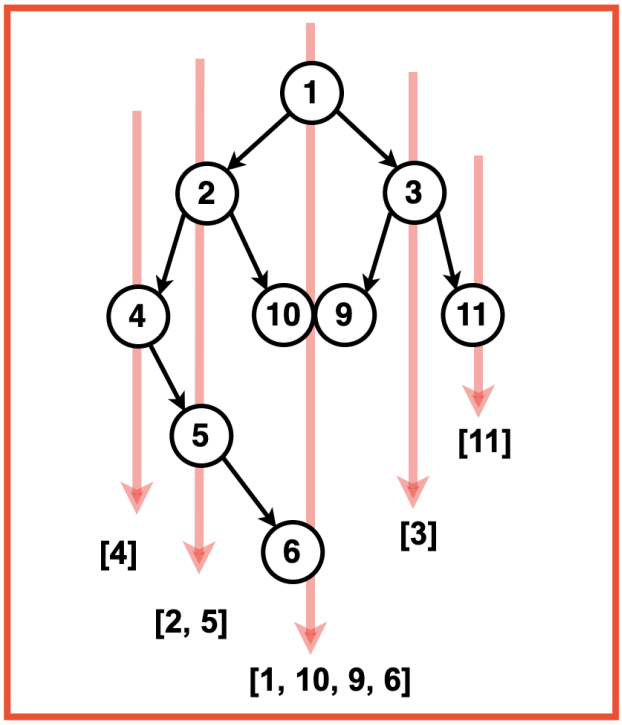
- Root lies at vertical level `0`
- Left child moves to vertical `-1`
- Right child moves to vertical +1
- Nodes having the same vertical value belong to the same column

If multiple nodes fall in the same vertical column, they must appear in **level order traversal order**.

Example 1
Binary Tree: `1 2 3 4 10 9 11 -1 5 -1 -1 -1 -1 -1 -1 -1 6`

Output:
`[[4], [2, 5], [1, 10, 9, 6], [3], [11]]`

Explanation:
Vertical -2 → [4]
Vertical -1 → [2, 5]
Vertical 0 → [1, 10, 9, 6]
Vertical 1 → [3]
Vertical 2 → [11]



Approach

Algorithm

We use **Level Order Traversal (BFS)** and track two coordinates for every node:

- x → vertical level
- y → tree level (depth)

Steps:

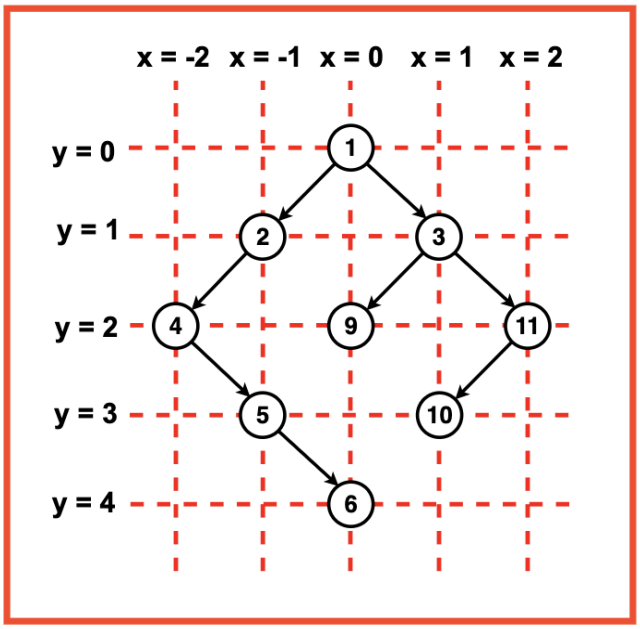
1. Use a map to store nodes based on vertical and level
   `map<int, map<int, multiset<int>>>`
2. Use a queue for BFS storing `(node, x, y)`
3. Start with root at `(0, 0)`
4. For every node:

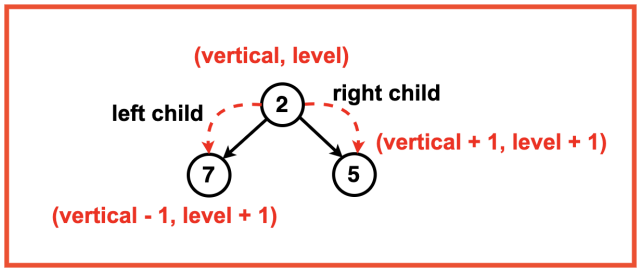
- Insert node value into map at `(x, y)`
- Left child → `(x-1, y+1)`
- Right child → `(x+1, y+1)`

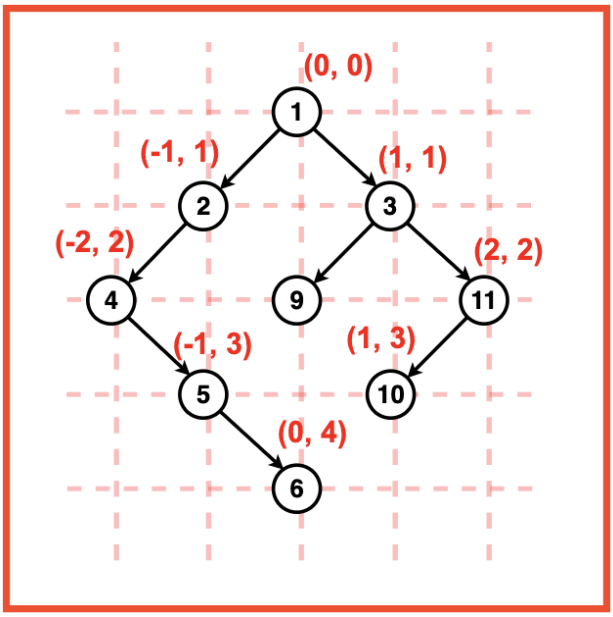
5. After BFS, traverse the map from leftmost vertical to rightmost
6. Collect values level by level into result

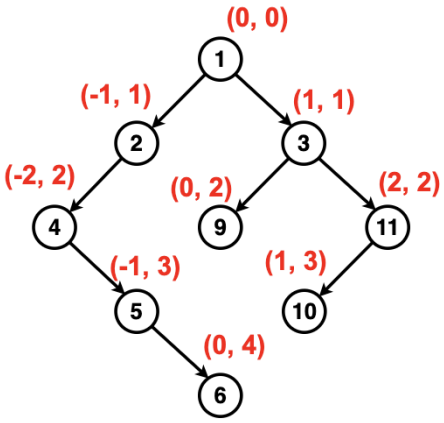
This guarantees:

- Left to right vertical order
- Top to bottom level order
- Correct order for overlapping nodes





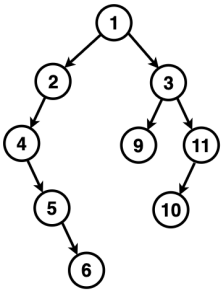




vertical [level] : [set of node values]

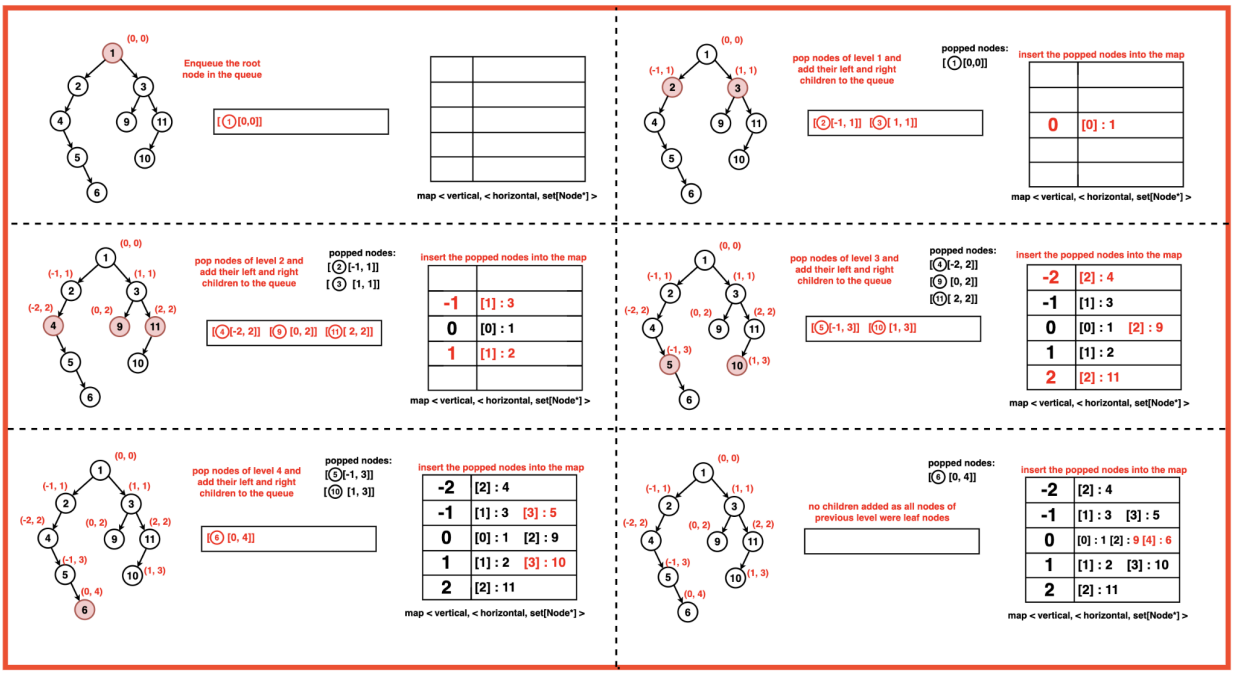
-2	[2] : 4
-1	[1] : 2, [3] : 5
0	[0] : 1, [2] : 9, [4] : 6
1	[1] : 3, [3] : 10
2	[2] : 11

map <int, map < int, multiset<int>>>



Queue <Node*, < vertical, horizontal>

map < vertical, < horizonatl, set[Node*] >



Code

```
#include <bits/stdc++.h>
using namespace std;

// Binary Tree Node
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    vector<vector<int>> verticalTraversal(Node* root) {
        // vertical -> level -> nodes
        map<int, map<int, multiset<int>>> nodes;
        queue<pair<Node*, pair<int,int>>> q;

        // root at vertical 0, level 0
        q.push({root, {0, 0}});
```

```
        while (!q.empty()) {
            auto temp = q.front();
            q.pop();

            Node* node = temp.first;
            int x = temp.second.first;
            int y = temp.second.second;

            nodes[x][y].insert(node->data);

            if (node->left)
                q.push({node->left, {x - 1, y + 1}});
            if (node->right)
                q.push({node->right, {x + 1, y + 1}});
        }

        vector<vector<int>> ans;

        for (auto vertical : nodes) {
            vector<int> col;
            for (auto level : vertical.second) {
                col.insert(col.end(),
                           level.second.begin(),
                           level.second.end());
            }
            ans.push_back(col);
        }
        return ans;
    }
};

int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(10);
```

```
    root->right->left = new Node(9);
    root->right->right = new Node(11);
    root->left->left->right = new Node(5);
    root->left->left->right->right = new Node(6);

    Solution sol;
    vector<vector<int>> res = sol.verticalTraversal(root);

    for (auto v : res) {
        cout << "[ ";
        for (int x : v) cout << x << " ";
        cout << "]";
    }
    return 0;
}
```

`multiset<int>`

- Values store karta hai
- Automatically sorted
- Duplicate allowed

👉 Ye isliye use hota hai kyunki:

- Same row & column me multiple nodes ho sakte hain
- Unhe sorted order me output karna hota hai

col.insert(col.end(),

level.second.begin(),

level.second.end());

level.second

multiset<int>

👉 Isme: same column & same row ke saare node values

already sorted order me hote hain

col.insert(...) ka matlab

col.insert(

col.end(), // end me insert karo

begin iterator, // kaha se

end iterator // kaha tak

);

👉 Matlab:

is row ke saare values ko col vector ke end me daal do

Complexity Analysis

Time Complexity: `O(N log N)`
Each node is visited once. Insertions into map and multiset take logarithmic time.

Space Complexity: `O(N)`
Map stores all nodes and queue can hold up to N/2 nodes in worst case.

17. Top View of a Binary Tree

Problem Statement
Given a Binary Tree, return its **Top View**.
The Top View of a Binary Tree contains the nodes that are visible when the tree is viewed from the top.

Question Explanation

To understand the top view, imagine looking at the binary tree from above.

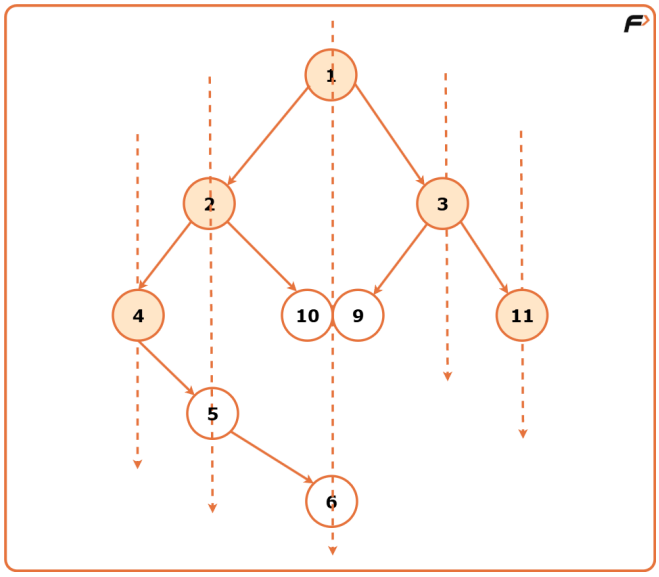
- Nodes are arranged on **vertical lines**
- Each node has a **vertical index**
    - Root → vertical = 0
    - Left child → vertical - 1
    - Right child → vertical + 1
- For every vertical line, **only the topmost node** (the one that appears first when moving level by level) will be visible

Example 1
Binary Tree: `1 2 3 4 10 9 11 -1 5 -1 -1 -1 -1 -1 -1 -1 6`

Top View Output: `[4, 2, 1, 3, 11]`

Explanation:

- Vertical -2 → 4
- Vertical -1 → 2
- Vertical 0 → 1
- Vertical +1 → 3
- Vertical +2 → 11



Approach

We use **Level Order Traversal (BFS)** because it ensures that nodes closer to the root are processed first.

1. If the tree is empty, return an empty array.
2. Create a map `mpp` where:
    - key → vertical index
    - value → first node seen at that vertical
3. Use a queue for BFS storing `(node, verticalIndex)`.
4. Start by pushing the root with vertical index `0`.
5. While the queue is not empty:
    - Pop the front node.
    - If its vertical index is **not present** in the map, store the node value.
    - Push left child with `vertical - 1`.

- Push right child with `vertical + 1`.

6. Traverse the map from leftmost to rightmost vertical and store values in the answer array.

7. Return the answer.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Node structure for Binary Tree
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    vector<int> topView(Node* root) {
        vector<int> ans;
        if (root == NULL) return ans;

        // Map to store first node at each vertical
        map<int, int> mpp;

        // Queue for BFS {node, vertical}
        queue<pair<Node*, int>> q;
        q.push({root, 0});

        while (!q.empty()) {
            auto it = q.front();
            q.pop();

            Node* node = it.first;
```

```
            int vertical = it.second;

            // Store only first node of each vertical
            if (mpp.find(vertical) == mpp.end()) {
                mpp[vertical] = node->data;
            }

            if (node->left)
                q.push({node->left, vertical - 1});
            if (node->right)
                q.push({node->right, vertical + 1});
        }

        // Collect result from map (already sorted)
        for (auto it : mpp) {
            ans.push_back(it.second);
        }

        return ans;
    }
};

// Driver code
int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(3);
    root->left->left = new Node(4);
    root->left->right = new Node(10);
    root->right->left = new Node(9);
    root->right->right = new Node(11);

    Solution sol;
    vector<int> result = sol.topView(root);

    cout << "Top View: ";
    for (int x : result) cout << x << " ";
    return 0;
```

}

Complexity Analysis

Time Complexity: `O(N)`
Each node is visited exactly once during BFS.

Space Complexity: `O(N)`
Queue and map together can store up to `N` nodes in the worst case.

18. Bottom View of a Binary Tree

Given a Binary Tree, return its **Bottom View**.
The Bottom View of a Binary Tree consists of the nodes that are visible when the tree is viewed from the bottom.

Question Explanation

To find the bottom view, imagine drawing **vertical lines** through the binary tree.

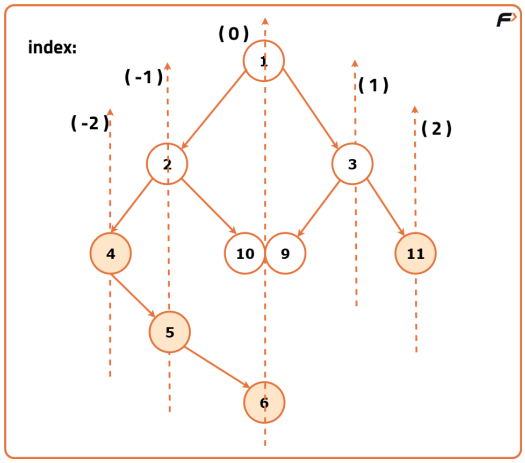
- Each node lies on a vertical line (called vertical index).
- Root node has vertical index `0`.
- Left child → vertical index `-1`.
- Right child → vertical index `+1`.
- For each vertical index, **the node that appears last in level order traversal** (i.e., the lowest node) will be visible from the bottom.

Example 1
Binary Tree: `1 2 3 4 10 9 11 -1 5 -1 -1 -1 -1 -1 -1 -1 6`

Bottom View Output: `[4, 5, 6, 3, 11]`

Explanation:

- Vertical -2 → 4
- Vertical -1 → 5
- Vertical 0 → 6
- Vertical +1 → 3
- Vertical +2 → 11



Approach 1 (Optimal Approach)

Algorithm

1. If the tree is empty, return an empty array.
2. Create a map where:
    - key → vertical index
    - value → node value at that vertical (updated continuously)
3. Use **level order traversal (BFS)** with a queue.
4. Push the root into the queue with vertical index `0`.
5. While the queue is not empty:
    - Pop a node and its vertical index.

- Update the map for this vertical index with the current node's value.
  (This ensures the bottom-most node overwrites previous ones.)
- Push left child with `vertical - 1`.
- Push right child with `vertical + 1`.

6. Traverse the map from leftmost to rightmost vertical index and store values in the result.
7. Return the result vector.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Node structure for the binary tree
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    vector<int> bottomView(Node* root) {
        vector<int> ans;
        if (root == NULL) return ans;

        // Map to store vertical -> bottom node value
        map<int, int> mpp;

        // Queue for BFS: {node, vertical index}
        queue<pair<Node*, int>> q;
        q.push({root, 0});

        while (!q.empty()) {
```

```
            auto it = q.front();
            q.pop();

            Node* node = it.first;
            int vertical = it.second;

            // Always overwrite to keep bottom-most node
            mpp[vertical] = node->data;

            if (node->left)
                q.push({node->left, vertical - 1});
            if (node->right)
                q.push({node->right, vertical + 1});
        }

        // Extract result from map
        for (auto it : mpp) {
            ans.push_back(it.second);
        }

        return ans;
    }
};

// Driver code
int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->left->left = new Node(4);
    root->left->right = new Node(10);
    root->left->left->right = new Node(5);
    root->left->left->right->right = new Node(6);
    root->right = new Node(3);
    root->right->left = new Node(9);
    root->right->right = new Node(11);

    Solution sol;
    vector<int> res = sol.bottomView(root);
```

```
    cout << "Bottom View: ";
    for (int x : res) cout << x << " ";
    return 0;
}
```

Complexity Analysis

Time Complexity: `O(N)`
Each node is visited exactly once during BFS traversal.

Space Complexity: `O(N)`
The queue and map together can store up to `N` nodes in the worst case.

19. Left View / Right View of a Binary Tree

Problem Recap

You are standing on **one side of a binary tree**:

- **Left View** → nodes visible from the **left side**
- **Right View** → nodes visible from the **right side**

Return the visible nodes **from top to bottom**.

At **each level**, **only one node is visible**:

- **Left View** → the **first node** encountered at that level
- **Right View** → the **first node** encountered at that level **when traversing right-first**

So the problem reduces to:

> "At each depth, pick the first node seen from the desired direction."

🔹 Optimal Approach

We use **Depth First Search** with level tracking.

Why DFS works?

- We track the **current level**
- If this level is visited for the **first time**, we store the node
- Order of recursion decides **left or right view**

✅ Left View (DFS)

Traversal Order

`Root → Left → Right`

Logic

- If `res.size() == level` → first node of this level → add it

Code (Left View)

```
#include <bits/stdc++.h>
using namespace std;
```

```
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    void dfsLeft(Node* root, int level, vector<int>& res) {
        if (!root) return;

        // First node at this level
        if (res.size() == level)
            res.push_back(root->data);

        dfsLeft(root->left, level + 1, res);
        dfsLeft(root->right, level + 1, res);
    }

    vector<int> leftView(Node* root) {
        vector<int> res;
        dfsLeft(root, 0, res);
        return res;
    }
};
```

✅ Right View (DFS)

Traversal Order

`Root → Right → Left`

Logic

- Same idea, but traverse **right first**

Code (Right View)

```
class Solution {
public:
    void dfsRight(Node* root, int level, vector<int>& res) {
        if (!root) return;

        if (res.size() == level)
            res.push_back(root->data);

        dfsRight(root->right, level + 1, res);
        dfsRight(root->left, level + 1, res);
    }

    vector<int> rightView(Node* root) {
        vector<int> res;
        dfsRight(root, 0, res);
        return res;
    }
};
```

🔹 BFS (Level Order) Approach (Also Valid)

Idea

- Perform level order traversal
- For each level:
    - **Left View** → pick first node
    - **Right View** → pick last node

BFS Code (Both Views)

```
vector<int> leftView(Node* root) {
    vector<int> res;
    if (!root) return res;

    queue<Node*> q;
    q.push(root);
```

```
    while (!q.empty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            Node* node = q.front(); q.pop();

            if (i == 0) res.push_back(node->data); // leftmost

            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
    }
    return res;
}

vector<int> rightView(Node* root) {
    vector<int> res;
    if (!root) return res;

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            Node* node = q.front(); q.pop();

            if (i == size - 1) res.push_back(node->data); // rightmost

            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
    }
    return res;
}
```

📊 Complexity Analysis

Approach	Time	Space
DFS	O(N)	O(H)
BFS	O(N)	O(N)

- N → number of nodes
- H → height of tree

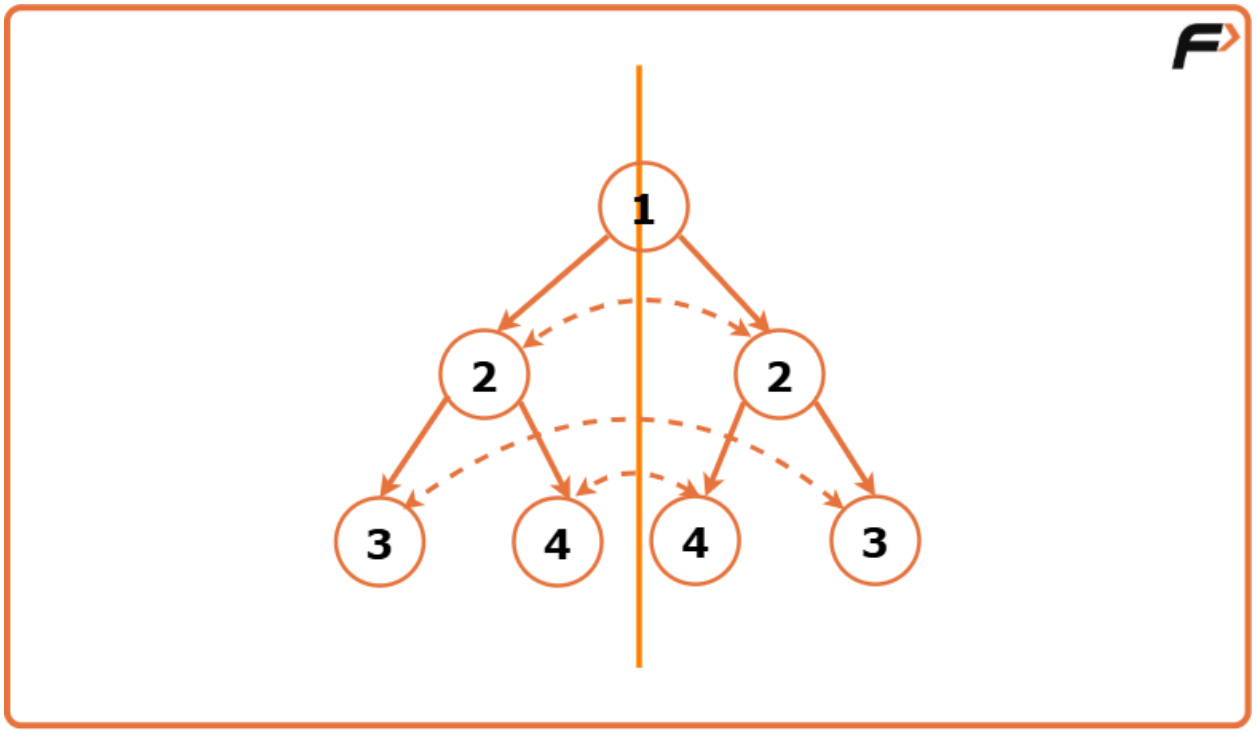
20. Check for Symmetrical Binary Tree

A Binary Tree is said to be **symmetric** if its left subtree is a mirror image of its right subtree.
If we draw a vertical line through the root, the structure and values on both sides should reflect each other exactly.

In other words:

- Left child of the left subtree must match the right child of the right subtree.
- Right child of the left subtree must match the left child of the right subtree.
- This condition must hold true for **every node** in the tree.

Example explanation:
For the tree 1 2 2 3 4 4 3
The left subtree `[2,3,4]` is a mirror of the right subtree `[2,4,3]`, so the tree is symmetric.



Approach

We solve this problem using **recursion** by comparing two subtrees in a mirrored manner.

Steps:

1. If the root is NULL, the tree is symmetric.
2. Create a helper function that takes two nodes:
    - One from the left subtree.
    - One from the right subtree.
3. Base cases:
    - If both nodes are NULL, return true.
    - If only one is NULL, return false.
4. Check:

- Values of both nodes must be equal.
- Left child of the first node must match the right child of the second node.
- Right child of the first node must match the left child of the second node.

5. If all checks pass recursively, the tree is symmetric.

This works because we always compare nodes that should mirror each other.

Code

```
#include <bits/stdc++.h>
using namespace std;

// Node structure for the binary tree
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

class Solution {
private:
    bool isSymmetricUtil(Node* root1, Node* root2) {
        if (root1 == NULL || root2 == NULL)
            return root1 == root2;

        return (root1->data == root2->data)
            && isSymmetricUtil(root1->left, root2->right)
            && isSymmetricUtil(root1->right, root2->left);
    }

public:
    bool isSymmetric(Node* root) {
        if (!root) return true;
        return isSymmetricUtil(root->left, root->right);
    }
};
```

```
int main() {
    Node* root = new Node(1);
    root->left = new Node(2);
    root->right = new Node(2);
    root->left->left = new Node(3);
    root->left->right = new Node(4);
    root->right->left = new Node(4);
    root->right->right = new Node(3);

    Solution sol;
    cout << sol.isSymmetric(root);
    return 0;
}
```

Complexity Analysis

Time Complexity: **O(N)**
Each node is visited once while comparing mirrored nodes.

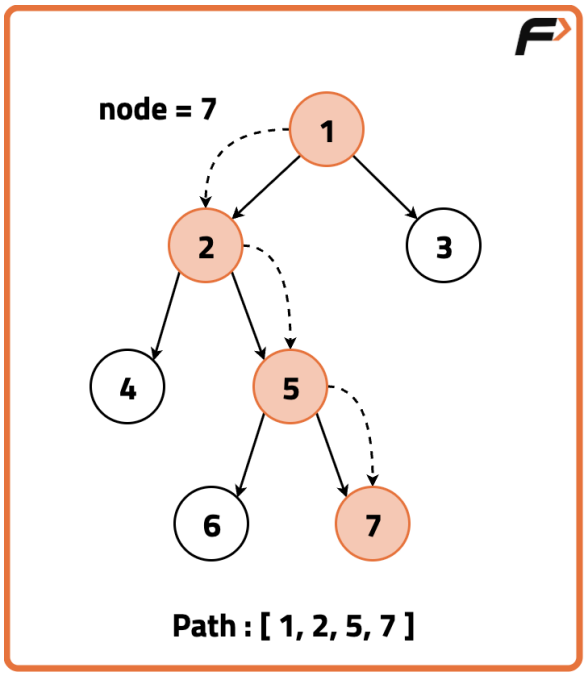
Space Complexity: **O(H)**
Extra space is used by the recursion stack, where H is the height of the tree.
In the worst case (skewed tree), H = N.

21. Print Root to Node Path in a Binary Tree

You are given:

- The **root** of a Binary Tree
- A **target node value** (unique and guaranteed to exist)

You need to **return the path from the root to that target node**.



Key Idea (DFS + Backtracking)

We perform a **Depth First Search (DFS)** and maintain a vector that stores the current path from the root.

At every node:

- Add the node to the path
- If this node is the target → path found
- Otherwise, search in **left** and **right** subtrees
- If the target is not found in either subtree, **backtrack** (remove the node from path)

This ensures that the path vector always represents the correct root-to-current-node path.

Algorithm

1. Start DFS from the root.

2. Push current node's value into the path vector.
3. If current node's value == target, return `true`.
4. Recursively search left and right subtrees.
5. If neither subtree contains the target:
    - Pop current node from path (backtracking)
    - Return `false`
6. Final path vector contains the root → target path.

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

// Structure for a binary tree node
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    // Helper DFS function
    bool getPath(TreeNode* root, vector<int>& path, int target) {
        if (root == NULL) return false;

        // Add current node
        path.push_back(root->val);

        // If target found
```

```
        if (root->val == target)
            return true;

        // Search left or right subtree
        if (getPath(root->left, path, target) ||
            getPath(root->right, path, target))
            return true;

        // Backtrack if target not found
        path.pop_back();
        return false;
    }

    // Main function
    vector<int> rootToNodePath(TreeNode* root, int target) {
        vector<int> path;
        getPath(root, path, target);
        return path;
    }
};

int main() {
    // Constructing the tree
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->left->right->right = new TreeNode(7);

    Solution sol;
    int target = 7;

    vector<int> path = sol.rootToNodePath(root, target);

    cout << "Path from root to node " << target << ": ";
    for (int x : path) cout << x << " ";
    return 0;
```

}

Example Walkthrough

For target 7, traversal path becomes:

1 → 2 → 5 → 7

Backtracking ensures that wrong branches are discarded automatically.

Complexity Analysis

- **Time Complexity:** `O(N)`
  Every node is visited once in the worst case.

- **Space Complexity:** `O(N)`
  Due to recursion stack + path vector.

22. Lowest Common Ancestor (LCA) of a Binary Tree

Given the **root** of a binary tree and **two nodes p and q**, find their **Lowest Common Ancestor (LCA)**.

> **LCA** is the **lowest (deepest) node** in the tree that has **both p and q as descendants**.
> A node can be a descendant of itself.

Key Insight (Recursive DFS)

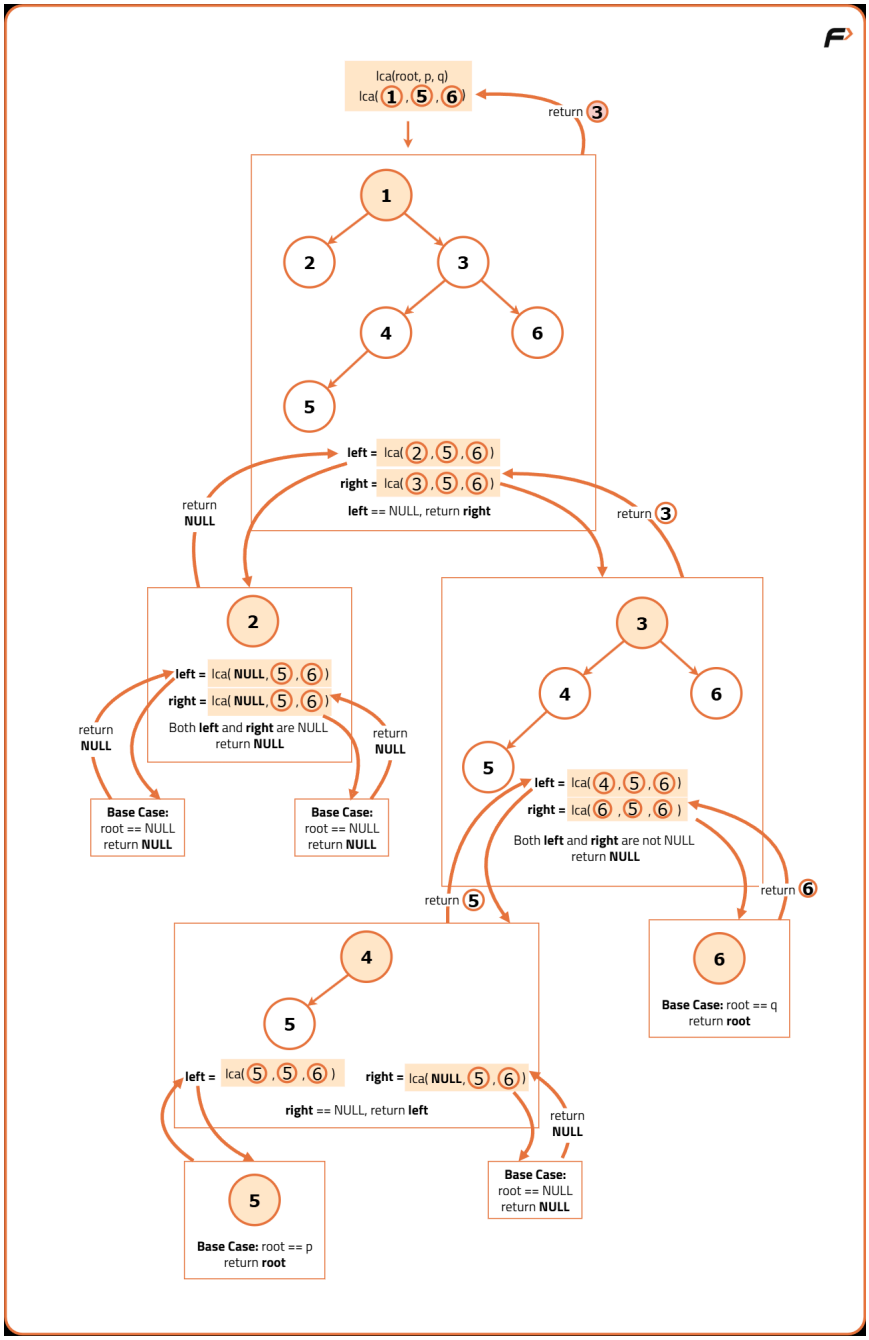
At every node, there are **three possibilities**:

1. p and q lie **in different subtrees** → current node is LCA
2. Both lie **in the left subtree**
3. Both lie **in the right subtree**

We solve this using **postorder traversal**.

Algorithm (Optimal & Clean)

1. **Base Case**
    - If `root == NULL`, return `NULL`
    - If `root == p` or `root == q`, return `root`
2. Recursively find LCA in:
    - Left subtree
    - Right subtree
3. **Decision**
    - If both left and right are non-null → current node is LCA
    - Else return the non-null child



C++ Code (Standard Interview Version)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int data;
```

```
    TreeNode* left;
    TreeNode* right;

    TreeNode(int val) : data(val), left(NULL), right(NULL) {}
};

class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p,
TreeNode* q) {
        // Base case
        if (root == NULL || root == p || root == q) {
            return root;
        }

        // Search left and right subtrees
        TreeNode* left = lowestCommonAncestor(root->left, p, q);
        TreeNode* right = lowestCommonAncestor(root->right, p, q);

        // If p and q are found in different subtrees
        if (left && right) {
            return root;
        }

        // Otherwise return non-null child
        return left ? left : right;
    }
};

// Driver Code
int main() {
    // Construct the tree
    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(5);
    root->right = new TreeNode(1);
    root->left->left = new TreeNode(6);
    root->left->right = new TreeNode(2);
    root->right->left = new TreeNode(0);
```

```
    root->right->right = new TreeNode(8);

    Solution sol;
    TreeNode* p = root->left;        // Node 5
    TreeNode* q = root->right;       // Node 1

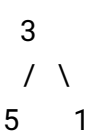
    TreeNode* lca = sol.lowestCommonAncestor(root, p, q);
    cout << "Lowest Common Ancestor: " << lca->data << endl;

    return 0;
}
```

Example Walkthrough

Example 1

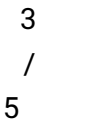
`p = 5, q = 1`



- 5 in left subtree
- 1 in right subtree
  ✅ **LCA = 3**

Example 2

`p = 5, q = 4`



```
\
 4
```

- One node is ancestor of the other
  ✅ **LCA = 5**

23. Maximum Width of a Binary Tree

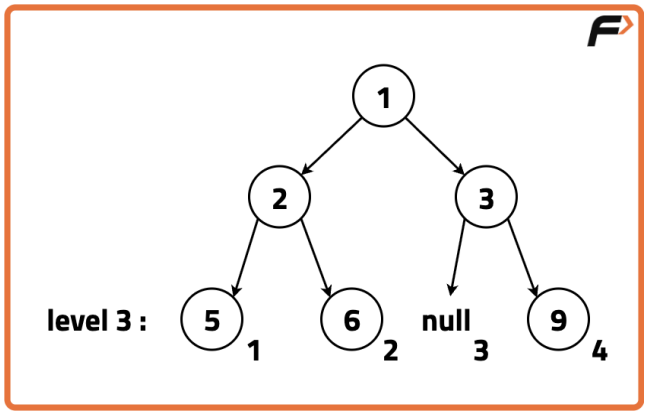
Given a Binary Tree, you need to find its **maximum width**.
The width of a level is defined as the number of positions between the **leftmost** and **rightmost** non-null nodes at that level, **including null positions in between**, assuming the tree is placed like a complete binary tree.

In simple words, for every level, imagine all nodes placed in their correct positions as in a complete binary tree. The width is the distance between the first and last node at that level. The answer is the **maximum such width among all levels**.

Example Explanation

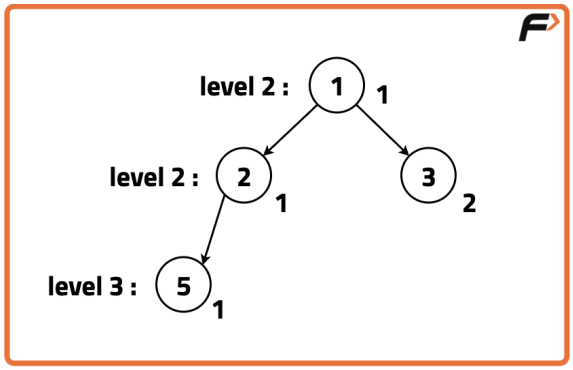
Example 1:
Binary Tree: `1 2 3 5 6 -1 9`

At level 3, the nodes look like:
`5 6 null 9`
Here, leftmost node is 5 and rightmost node is 9.
Even though there is a `null` in between, it is counted.
So width = 4, which is the maximum.



Example 2:
Binary Tree: 1 2 3 5

Level 2 contains nodes {2, 3}
Width = 2, which is the maximum.



Approach

We use **level order traversal (BFS)** and assign an index to every node as if the tree were a **complete binary tree**.

- Assign index `0` to the root.
- For any node with index `i`:
    - Left child gets index `2*i + 1`
    - Right child gets index `2*i + 2`
- For each level:
    - Note the index of the **first node** and **last node**
    - Width of that level = `lastIndex - firstIndex + 1`
- Keep updating the maximum width.

To avoid overflow, indices are **normalized** at every level by subtracting the minimum index of that level.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

class TreeNode {
public:
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) {
        val = x;
        left = nullptr;
        right = nullptr;
    }
};

class Solution {
public:
    int widthOfBinaryTree(TreeNode* root) {
        if (!root) return 0;

        int maxWidth = 0;
        queue<pair<TreeNode*, int>> q;
        q.push({root, 0});

        while (!q.empty()) {
            int size = q.size();
            int minIndex = q.front().second;
            int first = 0, last = 0;

            for (int i = 0; i < size; i++) {
                TreeNode* node = q.front().first;
                int currIndex = q.front().second - minIndex;
                q.pop();

                if (i == 0) first = currIndex;
```

```
                if (i == size - 1) last = currIndex;

                if (node->left)
                    q.push({node->left, 2 * currIndex + 1});
                if (node->right)
                    q.push({node->right, 2 * currIndex + 2});
            }

            maxWidth = max(maxWidth, last - first + 1);
        }
        return maxWidth;
    }
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(3);
    root->right = new TreeNode(2);
    root->left->left = new TreeNode(5);
    root->left->right = new TreeNode(3);
    root->right->right = new TreeNode(9);

    Solution sol;
    cout << "Maximum width: " << sol.widthOfBinaryTree(root);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)
Each node is visited exactly once during level order traversal.

Space Complexity: O(N)
The queue can hold up to $N/2$ nodes in the worst case (last level of the tree).

24. Check for Children Sum Property in a Binary Tree

Given a Binary Tree, you need to **convert** the tree so that it follows the **Children Sum Property**.

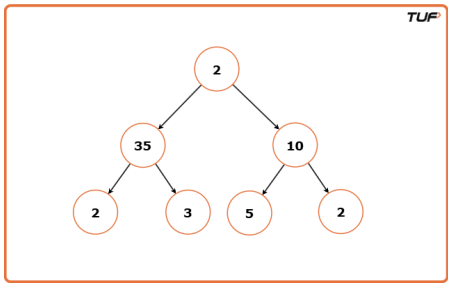
The Children Sum Property says that for **every node**,
the value of the node must be **equal to the sum of values of its left and right children**.

Important rules:

- If a child is missing, its value is considered `0`.
- You are **allowed to increase node values**, but **not allowed to decrease** any node value.
- The **structure of the tree must not change**.

Example Explanation

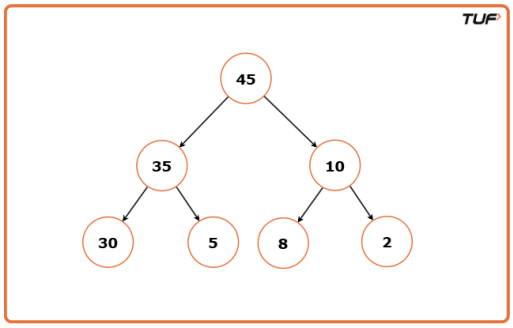
Example 1:
Input Tree values: `2 35 10 2 3 5 2`



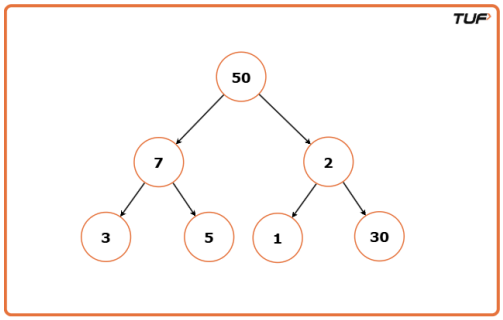
Some parent nodes have values greater than the sum of their children.
Since we cannot decrease parent values, we increase child values so that:

```
parent = left child + right child
```

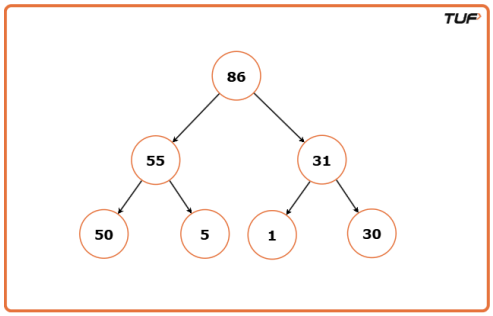
After valid increments, the tree becomes:
`45 35 10 30 5 8 2`



Example 2:
Input Tree values: `50 7 2 3 5 1 30`



Here, children sums are less than parent values at some nodes.
So we increase children values recursively until the property is satisfied.



Approach

- Traverse the tree using recursion.
- For each node:
    - Compute the sum of left and right child values.

    - If children sum is **greater than or equal to** node value, update node value.
    - If children sum is **less than** node value, increase one child value to match the parent.
- Recursively apply the same logic to left and right subtrees.
- After recursion, update the current node value to the final sum of its children.
- This ensures:
    - No node value is decreased.
    - Children Sum Property holds for all nodes.

Code (C++)

```
#include <iostream>
using namespace std;

struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    void changeTree(TreeNode* root) {
        if (root == NULL) {
            return;
        }

        int child = 0;
        if (root->left) {
            child += root->left->val;
        }
        if (root->right) {
```

```
            child += root->right->val;
        }

        if (child >= root->val) {
            root->val = child;
        } else {
            if (root->left) {
                root->left->val = root->val;
            } else if (root->right) {
                root->right->val = root->val;
            }
        }

        changeTree(root->left);
        changeTree(root->right);

        int tot = 0;
        if (root->left) {
            tot += root->left->val;
        }
        if (root->right) {
            tot += root->right->val;
        }

        if (root->left || root->right) {
            root->val = tot;
        }
    }
};

void inorderTraversal(TreeNode* root) {
    if (root == nullptr) return;
    inorderTraversal(root->left);
    cout << root->val << " ";
    inorderTraversal(root->right);
}

int main() {
```

```
    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(5);
    root->right = new TreeNode(1);
    root->left->left = new TreeNode(6);
    root->left->right = new TreeNode(2);
    root->right->left = new TreeNode(0);
    root->right->right = new TreeNode(8);
    root->left->right->left = new TreeNode(7);
    root->left->right->right = new TreeNode(4);

    Solution sol;

    cout << "Before: ";
    inorderTraversal(root);
    cout << endl;

    sol.changeTree(root);

    cout << "After: ";
    inorderTraversal(root);
    cout << endl;

    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)
Each node is visited once and processed in constant time.

Space Complexity: O(N)
Recursive call stack can go up to the height of the tree.

25. Print all the Nodes at a Distance of K in a Binary Tree

Given a Binary Tree, a target node, and an integer k, you need to return all nodes whose **distance from the target node is exactly k**.

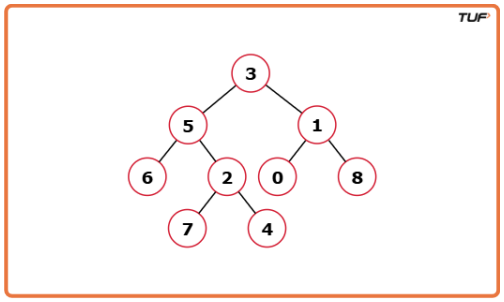
Distance means the number of edges between two nodes.
Nodes can be above the target (parent side) or below it (children side).
The result can be returned in **any order**.

Example Explanation

Example 1:
Tree: `[3,5,1,6,2,0,8,N,N,7,4]`, target = 5, k = 2

From node 5:

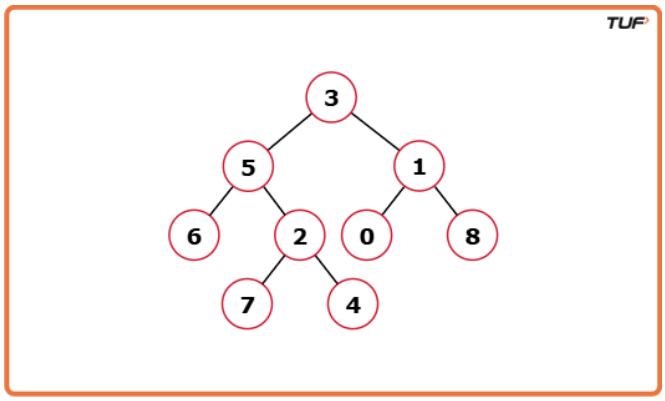
- Distance 1 nodes → 6, 2, 3
- Distance 2 nodes → 7, 4, 1



Output: `[7,4,1]`

Example 2:
Same tree, target = 5, k = 3

Distance 3 nodes from 5 → 0, 8



Output: `[0,8]`

Approach

- A binary tree only has child pointers, so we cannot move upward directly.
- First, create a **parent mapping** so every node knows its parent.
- This converts the tree into an **undirected graph**.
- Then perform **BFS starting from the target node**.
- Move in three directions from every node:
    - left child
    - right child
    - parent
- Use a `visited` set to avoid revisiting nodes.
- Stop BFS when distance k is reached.
- All nodes remaining in the queue are at distance k.

Steps:

1. Traverse the tree using BFS and store parent of each node.
2. Start BFS from the target node.
3. For each level, increase distance count.
4. When distance becomes k, collect node values and return.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

class Solution {
public:
    vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
        if (!root) return {};

        unordered_map<TreeNode*, TreeNode*> parentMap;
        buildParent(root, parentMap);

        queue<TreeNode*> q;
        unordered_set<TreeNode*> visited;

        q.push(target);
        visited.insert(target);

        int level = 0;

        while (!q.empty()) {
            int size = q.size();
```

```
            if (level == k) break;

            level++;

            for (int i = 0; i < size; i++) {
                TreeNode* node = q.front();
                q.pop();

                if (node->left && !visited.count(node->left)) {
                    visited.insert(node->left);
                    q.push(node->left);
                }

                if (node->right && !visited.count(node->right)) {
                    visited.insert(node->right);
                    q.push(node->right);
                }

                if (parentMap.count(node) &&
!visited.count(parentMap[node])) {
                    visited.insert(parentMap[node]);
                    q.push(parentMap[node]);
                }
            }
        }

        vector<int> ans;
        while (!q.empty()) {
            ans.push_back(q.front()->val);
            q.pop();
        }

        return ans;
    }

private:
    void buildParent(TreeNode* root, unordered_map<TreeNode*,
TreeNode*>& parentMap) {
```

```
        queue<TreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            TreeNode* node = q.front();
            q.pop();

            if (node->left) {
                parentMap[node->left] = node;
                q.push(node->left);
            }

            if (node->right) {
                parentMap[node->right] = node;
                q.push(node->right);
            }
        }
    }
};

int main() {
    TreeNode* root = new TreeNode(3);
    root->left = new TreeNode(5);
    root->right = new TreeNode(1);
    root->left->left = new TreeNode(6);
    root->left->right = new TreeNode(2);
    root->left->right->left = new TreeNode(7);
    root->left->right->right = new TreeNode(4);
    root->right->left = new TreeNode(0);
    root->right->right = new TreeNode(8);

    Solution sol;
    TreeNode* target = root->left;
    int k = 2;

    vector<int> result = sol.distanceK(root, target, k);

    for (int x : result) cout << x << " ";
```

```
    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)

- One BFS to build parent map.
- One BFS from target node.
- Each node is visited at most once.

Space Complexity: O(N)

- Parent map stores all nodes.
- Queue and visited set can store up to N nodes in worst case.

26. Minimum Time Taken to Burn the Binary Tree from a Node

You are given the root of a Binary Tree and a target node value.
If the target node is set on fire, the fire spreads every second to all directly connected nodes, which are: left child, right child, parent

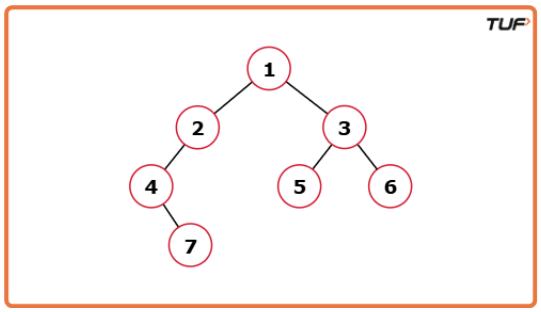
You have to find the **minimum time required to burn the entire binary tree**.

Example Explanation

Example 1:
Tree: `[1,2,3,4,null,5,6,null,7]`, target = 1

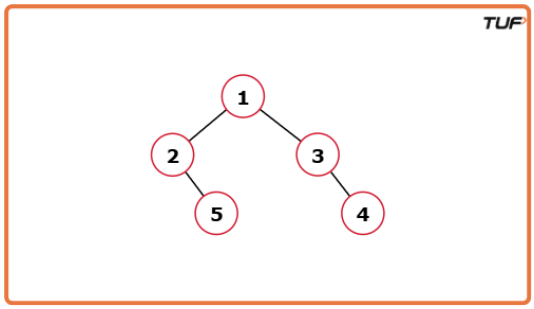
- Second 0: node 1 burns

- Second 1: nodes 2 and 3 burn
- Second 2: nodes 4 , 5 , 6 burn
- Second 3: node 7 burns



Output: 3

Example 2:
Tree: `[1,2,3,null,5,null,4]`, target = 4



- Second 0: node 4
- Second 1: node 3
- Second 2: node 1
- Second 3: node 2
- Second 4: node 5

Output: 4

Approach

- In a binary tree, nodes do not have parent pointers.

- But fire can spread **upwards** to parent also.
- So we first convert the binary tree into an **undirected graph**.
- Every node will be connected to:
    - its left child
    - its right child
    - its parent
- After conversion, the problem becomes:
    - starting from the target node, find how many seconds it takes to visit all nodes level by level.

Steps:

1. Traverse the tree and build an adjacency list (graph) using recursion.
2. Start BFS from the target node.
3. Use a visited set to avoid revisiting nodes.
4. Each BFS level represents **1 second of burning**.
5. Count how many levels are required until all nodes are burned.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};
```

```
class Solution {
public:
    int minTime(TreeNode* root, int target) {
        unordered_map<int, vector<int>> graph;
        buildGraph(root, NULL, graph);

        queue<int> q;
        unordered_set<int> visited;

        q.push(target);
        visited.insert(target);

        int time = 0;

        while (!q.empty()) {
            int size = q.size();
            bool spread = false;

            for (int i = 0; i < size; i++) {
                int node = q.front();
                q.pop();

                for (int nbr : graph[node]) {
                    if (!visited.count(nbr)) {
                        visited.insert(nbr);
                        q.push(nbr);
                        spread = true;
                    }
                }
            }

            if (spread) time++;
        }

        return time;
    }
```

```
private:
    void buildGraph(TreeNode* node, TreeNode* parent,
                    unordered_map<int, vector<int>>& graph) {
        if (!node) return;

        if (parent) {
            graph[node->val].push_back(parent->val);
            graph[parent->val].push_back(node->val);
        }

        buildGraph(node->left, node, graph);
        buildGraph(node->right, node, graph);
    }
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->right->left = new TreeNode(5);
    root->right->right = new TreeNode(6);
    root->left->left->right = new TreeNode(7);

    Solution sol;
    int target = 1;

    cout << "Minimum time to burn the tree: "
         << sol.minTime(root, target);

    return 0;
}
```

Complexity Analysis

Time Complexity: O(N)

- One traversal to build the graph.
- One BFS traversal to simulate burning.
- Each node is processed once.

Space Complexity: O(N)

- Adjacency list stores all nodes.
- Visited set and BFS queue can store up to N nodes.
- Hence linear extra space is used.

26. Count Number of Nodes in a Binary Tree

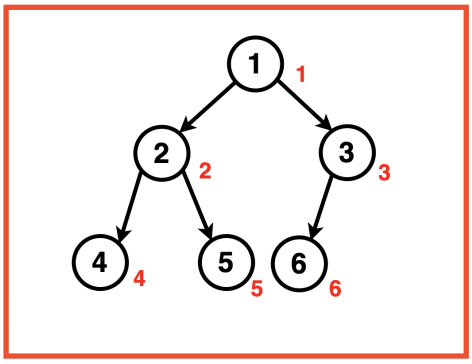
You are given the root of a **Complete Binary Tree**.
Your task is to count and return the total number of nodes in the tree.

A **Complete Binary Tree** is a tree where:

- All levels are completely filled except possibly the last.
- The last level nodes are filled from left to right.

Example Explanation

Example 1:
Tree: 1 2 3 4 5 6
All nodes are present level by level.
Total nodes = 6.



Example 2:
Tree: 2 4 3 5 9 8 7 1 6
All nodes follow the complete binary tree rule.
Total nodes = 9.

Approach 1: Brute Force Traversal

Algorithm

- Traverse the entire binary tree.
- Every time we visit a node, increase a counter by 1.
- We can use any traversal (inorder, preorder, postorder).
- Finally, return the counter value.

Steps:

1. Initialize `count = 0`.
2. Traverse the tree recursively.
3. For each non-null node, increment `count`.
4. Return `count`.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    void inorder(TreeNode* root, int &count) {
        if (root == NULL) return;
        count++;
        inorder(root->left, count);
        inorder(root->right, count);
    }

    int countNodes(TreeNode* root) {
        int count = 0;
        inorder(root, count);
        return count;
    }
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->right->left = new TreeNode(6);

    Solution sol;
    cout << sol.countNodes(root);
    return 0;
```

}

Complexity Analysis

Time Complexity: O(N)
Every node is visited once.

Space Complexity: O(N)
Recursive stack can go up to N in the worst case (skewed tree).

Approach 2: Optimal Using Complete Binary Tree Property

Algorithm

This approach uses the special property of a **complete binary tree**.

Key idea:

- If the left height and right height of a tree are equal, then the tree is a **perfect binary tree**.

A perfect binary tree with height h has:

```
nodes = 2^h - 1
```

-

Steps:

1. Find the height of the leftmost path.
2. Find the height of the rightmost path.
3. If both heights are equal:
    - Directly return `(2^height - 1)`.

4. Otherwise:
    - Recursively count nodes in left and right subtrees.
    - Return `1 + leftCount + rightCount`.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    int countNodes(TreeNode* root) {
        if (root == NULL) return 0;

        int lh = leftHeight(root);
        int rh = rightHeight(root);

        if (lh == rh) {
            return (1 << lh) - 1;  //2^h -1
        }

        return 1 + countNodes(root->left) + countNodes(root->right);
    }

    int leftHeight(TreeNode* node) {
        int h = 0;
        while (node) {
            h++;
```

```
            node = node->left;
        }
        return h;
    }

    int rightHeight(TreeNode* node) {
        int h = 0;
        while (node) {
            h++;
            node = node->right;
        }
        return h;
    }
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->right->left = new TreeNode(6);

    Solution sol;
    cout << sol.countNodes(root);
    return 0;
}
```

Complexity Analysis

Time Complexity: O(log N × log N)

- Height calculation takes `O(log N)`.
- Recursive calls happen at most `log N` times.

Space Complexity: O(log N)

- Recursion depth equals height of the tree.
- Since the tree is complete, height is `log N`.

27. Requirements needed to construct a Unique Binary Tree | Theory

You are given **two traversal types** of a binary tree, and you need to decide **whether these two traversals are sufficient to construct a UNIQUE binary tree**.

Each traversal is represented by a number:

- 1 → Preorder
- 2 → Inorder
- 3 → Postorder

The task is **not to construct the tree**, but only to check **if a unique binary tree is possible** from the given pair of traversals.

Key Idea (Very Important)

A **binary tree can be uniquely constructed only if one of the traversals is INORDER**, combined with either:

- Preorder
- Postorder

If **Inorder traversal is missing**, then the tree **cannot be uniquely determined**.

Why Inorder is Necessary

- **Inorder traversal** gives the exact position of the root between left and right subtrees.
- Preorder tells us the root first.
- Postorder tells us the root last.
- Without inorder, we cannot decide how nodes are split into left and right subtrees.

So:

- Inorder + Preorder → Unique tree
- Inorder + Postorder → Unique tree
- Preorder + Postorder → **Not unique**

Valid Traversal Combinations

Traversal Pair	Unique Tree Possible?	Reason
Preorder (1) + Inorder (2)	Yes	Inorder gives left/right split
Inorder (2) + Preorder (1)	Yes	Same as above
Postorder (3) + Inorder (2)	Yes	Inorder gives structure
Inorder (2) + Postorder (3)	Yes	Same as above

Preorder (1) + Postorder (3)	No	Multiple trees possible
Postorder (3) + Preorder (1)	No	Multiple trees possible

Example Explanation

Example 1

Input: `a = 1 (Preorder), b = 2 (Inorder)`
Output: `true`

Explanation:
Preorder gives root order, inorder gives exact left-right separation.
So, a unique binary tree can be constructed.

Example 2

Input: `a = 1 (Preorder), b = 3 (Postorder)`
Output: `false`

Explanation:
Without inorder traversal, we cannot uniquely determine the structure.
Multiple binary trees are possible with the same preorder and postorder.

Final Rule to Remember (Interview Ready)

> **A unique binary tree can be constructed if and only if one of the traversals is INORDER.**

This logic works in **O(1) time and O(1) space**, since we are only checking traversal types, not building the tree.

28. Construct A Binary Tree from Inorder and Preorder Traversal

You are given two arrays:

- **Preorder traversal** of a binary tree
- **Inorder traversal** of the same binary tree

Your task is to **construct the unique binary tree** represented by these two traversals and return its root.

A unique binary tree is always possible when **Preorder + Inorder** traversals are given.

Question Explanation

- **Preorder traversal** visits nodes in this order:
  `Root → Left → Right`

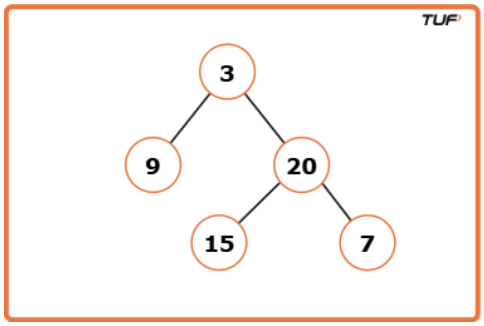
- **Inorder traversal** visits nodes in this order:
  `Left → Root → Right`

From preorder, we always know **which node is the root**.
From inorder, we know **how the tree is split into left and right subtrees**.

By combining both traversals, we can recursively build the exact tree.

Example Explanation

Input
preorder = `[3, 9, 20, 15, 7]`
inorder = `[9, 3, 15, 20, 7]`



Steps:

- First element of preorder is 3 → root
- In inorder, 3 splits array into:
    - Left subtree → `[9]`
    - Right subtree → `[15, 20, 7]`
- Recursively repeat the same steps for left and right parts

This process builds the unique binary tree.

Approach

1. Store the index of each value from inorder traversal in a map for O(1) lookup.
2. Use a recursive function with index ranges instead of slicing arrays.
3. Base Case:
    - If preorder range or inorder range becomes invalid, return `NULL`.
4. Take the first element of the current preorder range as the root.
5. Find the root's index in inorder using the map.
6. Calculate the number of nodes in the left subtree.

7. Recursively construct:
    - Left subtree using left part of inorder
    - Right subtree using right part of inorder
8. Return the constructed root.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// TreeNode structure
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        map<int,int> inMap;
        for(int i=0;i<inorder.size();i++){
```

```
            inMap[inorder[i]] = i;
        }
        return build(preorder,0,preorder.size()-1,
                     inorder,0,inorder.size()-1,inMap);
    }

private:
    TreeNode* build(vector<int>& preorder,int preStart,int preEnd,
                    vector<int>& inorder,int inStart,int inEnd,
                    map<int,int>& inMap){
        if(preStart > preEnd || inStart > inEnd) return nullptr;

        TreeNode* root = new TreeNode(preorder[preStart]);
        int inRoot = inMap[root->val];
        int leftSize = inRoot - inStart;

        root->left = build(preorder,preStart+1,preStart+leftSize,
                           inorder,inStart,inRoot-1,inMap);
        root->right = build(preorder,preStart+leftSize+1,preEnd,
                            inorder,inRoot+1,inEnd,inMap);
        return root;
    }
};
```

```
// Inorder print for verification
void printInorder(TreeNode* root){
    if(!root) return;
    printInorder(root->left);
    cout<<root->val<<" ";
    printInorder(root->right);
}

int main(){
    vector<int> preorder = {3,9,20,15,7};
    vector<int> inorder = {9,3,15,20,7};

    Solution sol;
    TreeNode* root = sol.buildTree(preorder,inorder);

    printInorder(root);
    return 0;
}
```

Time Complexity:
O(N)
Each node is created and processed exactly once.

Space Complexity:
`O(N)`

- Hash map stores N elements
- Recursion stack can go up to height of tree (worst case N)

29. Construct Binary Tree from Inorder and Postorder Traversal

You are given two arrays:

- **Inorder traversal** of a binary tree
- **Postorder traversal** of the same binary tree

Your task is to **construct the unique binary tree** represented by these two traversals and return its root.

A unique binary tree can always be constructed when **Inorder + Postorder** traversals are given.

- **Inorder traversal** follows:
  `Left → Root → Right`
- **Postorder traversal** follows:
  `Left → Right → Root`

From **postorder**, the **last element is always the root** of the current tree or subtree.
From **inorder**, once we know the root, we can split the tree into:

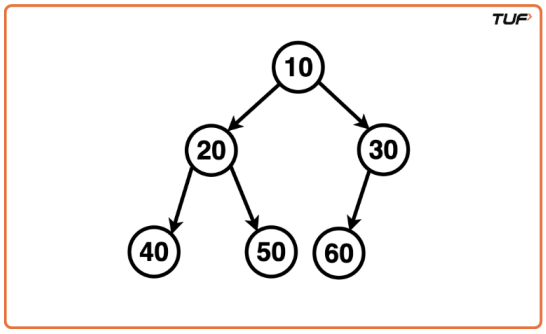
- Left subtree (elements before root)

- Right subtree (elements after root)

Using this idea recursively, the entire tree can be constructed.

Example Explanation

Input
Inorder = `[40, 20, 50, 10, 60, 30]`
Postorder = `[40, 50, 20, 60, 30, 10]`



Steps:

1. Last element of postorder is `10` → root
2. In inorder, `10` splits array into:
    - Left: `[40, 20, 50]`
    - Right: `[60, 30]`
3. Recursively repeat the same steps for left and right parts

This process builds the unique binary tree.

Approach

1. Create a hashmap to store the index of each element in the inorder traversal for fast lookup.
2. Define a recursive function that works with index ranges instead of slicing arrays.
3. Base Case:
    - If inorder or postorder range becomes invalid, return `NULL`.
4. Take the **last element of the current postorder range** as the root.
5. Find the root's index in inorder using the hashmap.
6. Calculate the size of the left subtree.
7. Recursively build:
    - Left subtree using corresponding inorder and postorder ranges
    - Right subtree using remaining ranges
8. Return the constructed root.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// TreeNode structure
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder)
{
        map<int,int> inMap;
```

```
        for(int i=0;i<inorder.size();i++){
            inMap[inorder[i]] = i;
        }
        return build(inorder,0,inorder.size()-1,
                     postorder,0,postorder.size()-1,inMap);
    }

private:
    TreeNode* build(vector<int>& inorder,int inStart,int inEnd,
                    vector<int>& postorder,int postStart,int postEnd,
                    map<int,int>& inMap){
        if(inStart > inEnd || postStart > postEnd) return nullptr;

        TreeNode* root = new TreeNode(postorder[postEnd]);
        int inRoot = inMap[root->val];
        int leftSize = inRoot - inStart;

        root->left = build(inorder,inStart,inRoot-1,

postorder,postStart,postStart+leftSize-1,inMap);

        root->right = build(inorder,inRoot+1,inEnd,

postorder,postStart+leftSize,postEnd-1,inMap);
        return root;
    }
};

// Inorder print to verify tree
void printInorder(TreeNode* root){
    if(!root) return;
    printInorder(root->left);
    cout<<root->val<<" ";
    printInorder(root->right);
}

int main(){
    vector<int> inorder = {40,20,50,10,60,30};
```

```
    vector<int> postorder = {40,50,20,60,30,10};

    Solution sol;
    TreeNode* root = sol.buildTree(inorder,postorder);

    printInorder(root);
    return 0;
}
```

Complexity Analysis

Time Complexity:
O(N)
Each node is created and processed exactly once.

Space Complexity:
O(N)

- Hashmap stores N elements
- Recursion stack can go up to height of tree (worst case N)

30. Serialize And Deserialize a Binary Tree

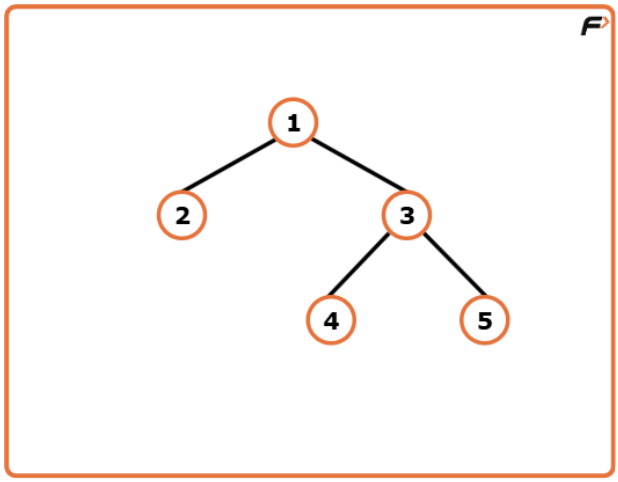
Given a Binary Tree, you need to **serialize** it into a string and then **deserialize** that string back into the original binary tree structure.

- **Serialization** means converting the tree into a string format that can be stored or transmitted.
- **Deserialization** means reconstructing the exact same binary tree from that string.

There is no restriction on the format, but after deserialization, the tree structure must remain exactly the same as the original.

Example Explanation

Input Tree:
`1 2 3 -1 -1 4 5`



Using **level order traversal**:

- We store node values in order
- Use # to represent NULL nodes
- Separate values using commas

Serialized String:
`1,2,3,#,#,4,5,#,#,#,#,`

During deserialization, we read this string and rebuild the tree level by level, reconnecting left and right children correctly.

Algorithm

Serialization (Level Order Traversal)

1. If root is NULL, return empty string.
2. Use a queue for BFS.
3. Push root into the queue.
4. While queue is not empty:
    - Pop a node.
    - If node is NULL, append `"#,"` to string.
    - Else append `node->val + ","` and push its left and right children.
5. Return the final string.

Deserialization (Level Order Reconstruction)

1. If input string is empty, return NULL.
2. Use `stringstream` to split values by comma.
3. First value is the root.
4. Use a queue to rebuild tree level by level.
5. For each node:
    - Read left value → if not #, create left child.
    - Read right value → if not #, create right child.
6. Continue until string is fully processed.
7. Return the root.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;
```

```
// Definition for a binary tree node
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    // Serialize the tree using level order traversal
    string serialize(TreeNode* root) {
        if (!root) return "";

        string s = "";
        queue<TreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            TreeNode* node = q.front();
            q.pop();

            if (node == nullptr) {
                s += "#,";
            } else {
                s += to_string(node->val) + ",";
                q.push(node->left);
                q.push(node->right);
            }
        }
        return s;
    }

    // Deserialize the string back to tree
    TreeNode* deserialize(string data) {
        if (data.size() == 0) return nullptr;
```

```
        stringstream ss(data);
        string str;
        getline(ss, str, ',');

        TreeNode* root = new TreeNode(stoi(str));
        queue<TreeNode*> q;
        q.push(root);

        while (!q.empty()) {
            TreeNode* node = q.front();
            q.pop();

            getline(ss, str, ',');
            if (str != "#") {
                node->left = new TreeNode(stoi(str));
                q.push(node->left);
            }

            getline(ss, str, ',');
            if (str != "#") {
                node->right = new TreeNode(stoi(str));
                q.push(node->right);
            }
        }
        return root;
    }
};

// Inorder traversal for checking
void inorder(TreeNode* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->val << " ";
    inorder(root->right);
}

int main() {
    TreeNode* root = new TreeNode(1);
```

```
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->right->left = new TreeNode(4);
    root->right->right = new TreeNode(5);

    Solution sol;

    string serialized = sol.serialize(root);
    cout << "Serialized: " << serialized << endl;

    TreeNode* deserialized = sol.deserialize(serialized);
    cout << "Tree after deserialization (Inorder): ";
    inorder(deserialized);
    cout << endl;

    return 0;
}
```

Complexity Analysis

Time Complexity: `O(N)`

- Serialization visits each node once.
- Deserialization also processes each node once.

Space Complexity: `O(N)`

- Queue stores nodes during BFS.
- Serialized string also stores all node values including NULL markers.

31. Morris Preorder Traversal of a Binary Tree

Given a Binary Tree, you need to perform **Morris Preorder Traversal** and return the preorder sequence of the tree.

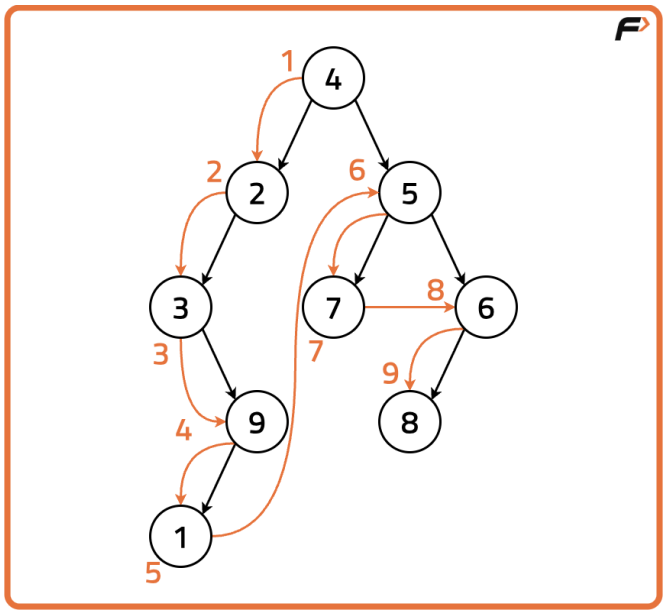
Preorder traversal follows the order:
Root → Left → Right

The key requirement of Morris Traversal is that it must be done in **O(1) extra space**, which means:

- No recursion
- No stack
- Only temporary modification of tree pointers (which are restored later)

Example Explanation

Input Tree:
1 2 3 4 5 6



Using Morris Preorder Traversal:

- We visit the root first
- Then traverse the left subtree
- Finally traverse the right subtree
- Temporary links (threads) are created to avoid recursion and stack usage

Output:

```
[1, 2, 4, 5, 6, 3]
```

Algorithm

This algorithm is based on **Morris Inorder Traversal**, with a small change to achieve **Preorder traversal**.

1. Initialise a pointer `cur` pointing to the root.
2. While `cur` is not NULL:
    - If `cur->left` is NULL:
        - Add `cur->val` to the preorder list.
        - Move to `cur->right`.
    - Else:
        - Find the inorder predecessor of `cur` (rightmost node in left subtree).
        - If predecessor's right is NULL:
            - Create a temporary thread from predecessor to `cur`.
            - Move `cur` to its left child.
        - Else:

- Remove the temporary thread.
- Add `cur->val` to the preorder list.
- Move `cur` to its right child.

3. Continue until the entire tree is traversed.
4. Return the preorder list.

This traversal modifies the tree temporarily but restores it before moving ahead.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// TreeNode structure
struct TreeNode {
    int val;
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    vector<int> getPreorder(TreeNode* root) {
    vector<int> preorder;
    TreeNode* cur = root;

    while (cur != NULL) {
        if (cur->left == NULL) {
            preorder.push_back(cur->val);
            cur = cur->right;
        } else {
            TreeNode* prev = cur->left;
            while (prev->right && prev->right != cur) {
```

```
                prev = prev->right;
            }

            if (prev->right == NULL) {
                preorder.push_back(cur->val);
                prev->right = cur;
                cur = cur->left;
            } else {
                prev->right = NULL;
                cur = cur->right;
            }
        }
    }
    return preorder;
}


};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->left->right->right = new TreeNode(6);

    Solution sol;
    vector<int> preorder = sol.getPreorder(root);

    cout << "Binary Tree Morris Preorder Traversal: ";
    for (int x : preorder) cout << x << " ";
    cout << endl;

    return 0;
}
```

Time Complexity: $O(2N)$
Each node is visited at most twice:

- Once when creating the temporary thread
- Once when removing the thread

Hence overall time is linear.

Space Complexity: $O(1)$
No recursion or stack is used. Only constant extra pointers are required.

32. Morris Inorder Traversal of a Binary Tree

Given a Binary Tree, implement **Morris Inorder Traversal** and return the inorder sequence of the tree.

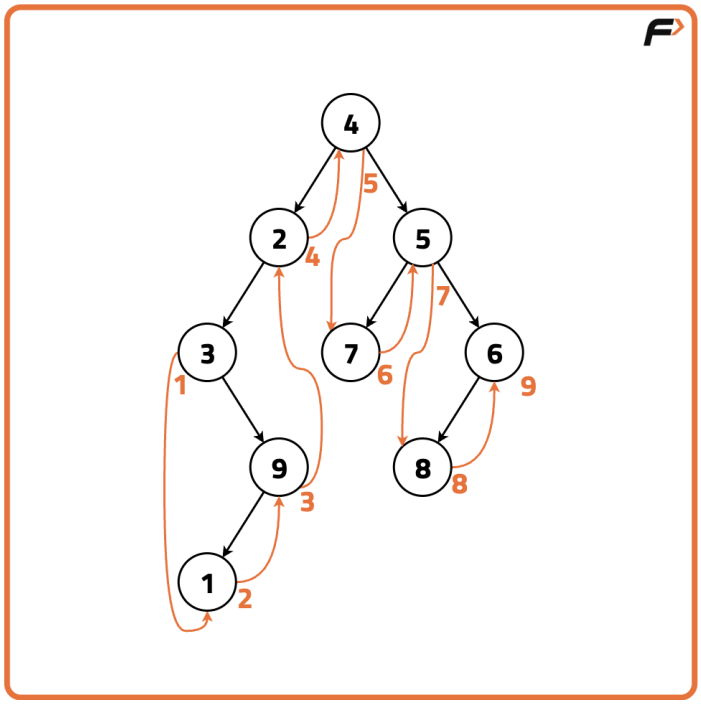
Inorder traversal follows the order:
Left → Root → Right

Morris Traversal is special because it performs this traversal using **O(1) extra space**, which means:

- No recursion
- No stack
- The tree structure is temporarily modified and then restored

---

Input Tree:
1 2 3 4 5 6



Inorder traversal visits:

- Left subtree first
- Then the root
- Then the right subtree

Output:
`[4, 2, 5, 6, 1, 3]`

Morris traversal achieves this without storing nodes externally.

Approach:

Algorithm

Morris Inorder Traversal uses the concept of an **inorder predecessor**.

1. Initialise a pointer `cur` pointing to the root.

2. While `cur` is not NULL:
    - **Case 1:** If `cur->left` is NULL
        - Add `cur->val` to result.
        - Move `cur` to `cur->right`.
    - **Case 2:** If `cur->left` is not NULL
        - Find inorder predecessor (rightmost node in left subtree).
        - If predecessor's right is NULL:
            - Create a temporary link from predecessor to `cur`.
            - Move `cur` to its left child.
        - Else:
            - Remove the temporary link.
            - Add `cur->val` to result.
            - Move `cur` to its right child.
3. Continue until the entire tree is traversed.
4. Return the inorder sequence.

Temporary links ensure we return back after finishing the left subtree.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

// TreeNode structure
struct TreeNode {
    int val;
```

```
    TreeNode *left;
    TreeNode *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    vector<int> getInorder(TreeNode* root) {
        vector<int> inorder;
        TreeNode* cur = root;

        while (cur != NULL) {
            if (cur->left == NULL) {
                inorder.push_back(cur->val);
                cur = cur->right;
            } else {
                TreeNode* prev = cur->left;
                while (prev->right && prev->right != cur) {
                    prev = prev->right;
                }

                if (prev->right == NULL) {
                    prev->right = cur;
                    cur = cur->left;
                } else {
                    prev->right = NULL;
                    inorder.push_back(cur->val);
                    cur = cur->right;
                }
            }
        }
        return inorder;
    }
};

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
```

```
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->left->right->right = new TreeNode(6);

    Solution sol;
    vector<int> inorder = sol.getInorder(root);

    cout << "Binary Tree Morris Inorder Traversal: ";
    for (int x : inorder) cout << x << " ";
    cout << endl;

    return 0;
}
```

Complexity Analysis

Time Complexity: $O(2N)$
Each node is visited at most twice:

- Once when creating the temporary thread
- Once when removing it

So overall time is linear.

Space Complexity: $O(1)$
No recursion, no stack, only constant extra pointers are use

33. Flatten Binary Tree to Linked List

Given a Binary Tree, convert it into a **linked list in-place** such that the nodes follow the **preorder traversal order** of the tree.

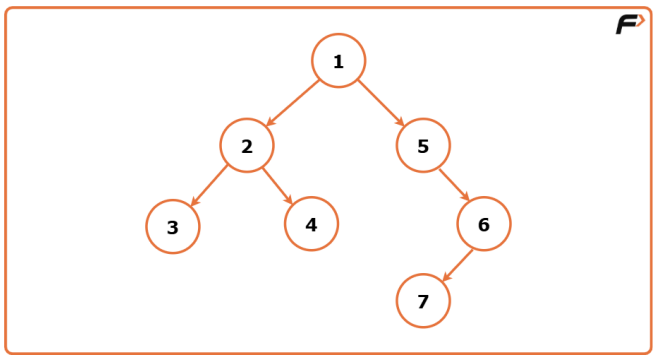
Rules to follow:

- Use the **right pointer** as the `next` pointer of the linked list.
- Set **all left pointers to NULL**.
- Do **not create new nodes**.
- Modify the tree **in-place**.

Preorder traversal order is:
Root → Left → Right

Example Explanation

Input Tree:
`1 2 5 3 4 -1 6 -1 -1 -1 -1 7`



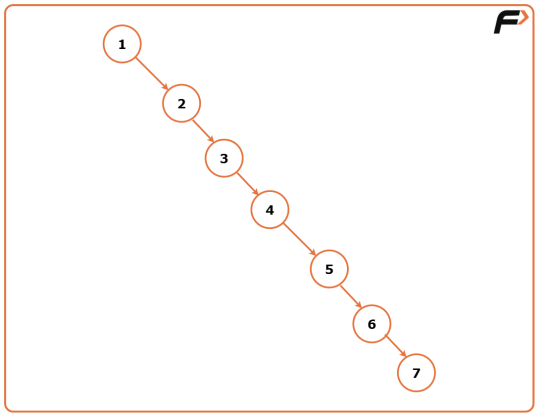
Preorder traversal:
`[1, 2, 3, 4, 5, 6, 7]`

After flattening:

- Each node's left = NULL

- Each node's right points to the next node in preorder

Output (Right pointers):
1 → 2 → 3 → 4 → 5 → 6 → 7



Approach 1: Brute Force (Recursive – Reverse Preorder)

Algorithm

1. Use a global pointer `prev` to store the previously processed node.
2. Traverse the tree in **reverse preorder**:
    - Right subtree
    - Left subtree
3. For each node:
    - Set `root->right = prev`
    - Set `root->left = NULL`
    - Update `prev = root`
4. This builds the linked list in correct preorder order when recursion unwinds.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    TreeNode* prev = nullptr;

    void flatten(TreeNode* root) {
        if (root == nullptr) return;

        flatten(root->right);
        flatten(root->left);

        root->right = prev;
        root->left = nullptr;
        prev = root;
    }
};

void printFlattenTree(TreeNode* root) {
    while (root) {
        cout << root->val << " ";
        root = root->right;
    }
}

int main() {
    TreeNode* root = new TreeNode(1);
```

```
    root->left = new TreeNode(2);
    root->right = new TreeNode(5);
    root->left->left = new TreeNode(3);
    root->left->right = new TreeNode(4);
    root->right->right = new TreeNode(6);

    Solution sol;
    sol.flatten(root);

    printFlattenTree(root);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each node is visited once.

- **Space Complexity:** `O(H)`
  Due to recursion stack, where H is tree height.

Approach 2: Better Approach (Iterative Using Stack)

Algorithm

1. Use a stack to simulate preorder traversal.
2. Push root into stack.
3. While stack is not empty:
    - Pop the top node.
    - Push right child (if exists).
    - Push left child (if exists).

- Set current node’s right to stack top.
- Set current node’s left to NULL.

4. This directly links nodes in preorder order.

Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    void flatten(TreeNode* root) {
        if (root == nullptr) return;

        stack<TreeNode*> st;
        st.push(root);

        while (!st.empty()) {
            TreeNode* cur = st.top();
            st.pop();

            if (cur->right) st.push(cur->right);
            if (cur->left) st.push(cur->left);

            if (!st.empty()) cur->right = st.top();
            cur->left = nullptr;
        }
    }
};
```

```
void printFlattenTree(TreeNode* root) {
    while (root) {
        cout << root->val << " ";
        root = root->right;
    }
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(5);
    root->left->left = new TreeNode(3);
    root->left->right = new TreeNode(4);
    root->right->right = new TreeNode(6);

    Solution sol;
    sol.flatten(root);

    printFlattenTree(root);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** `O(N)`
  Each node is processed once.

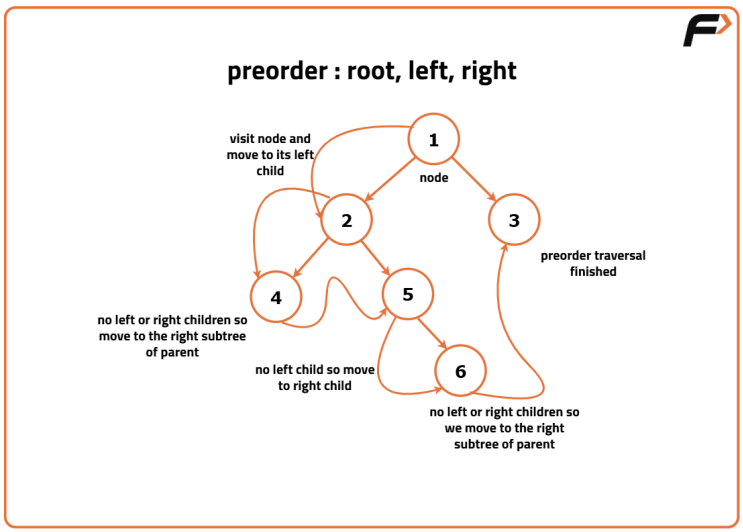
- **Space Complexity:** `O(H)`
  Stack space depends on tree height.

Approach 3: Optimal Approach (Morris Traversal Style)

1. Start with `curr = root`.

2. While `curr` is not NULL:
    - If `curr->left` exists:
        - Find the rightmost node in left subtree.
        - Connect it to `curr->right`.
        - Move left subtree to right.
        - Set `curr->left = NULL`.
    - Move `curr = curr->right`.
3. Tree becomes a right-skewed linked list in preorder.

This modifies pointers in-place without recursion or stack.



Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
```

```
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    void flatten(TreeNode* root) {
        TreeNode* curr = root;

        while (curr) {
            if (curr->left) {
                TreeNode* pre = curr->left;
                while (pre->right) {
                    pre = pre->right;
                }
                pre->right = curr->right;
                curr->right = curr->left;
                curr->left = nullptr;
            }
            curr = curr->right;
        }
    }
};

void printFlattenTree(TreeNode* root) {
    while (root) {
        cout << root->val << " ";
        root = root->right;
    }
}

int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(5);
    root->left->left = new TreeNode(3);
    root->left->right = new TreeNode(4);
    root->right->right = new TreeNode(6);
```

```
    Solution sol;
    sol.flatten(root);

    printFlattenTree(root);
    return 0;
}
```

Complexity Analysis

- **Time Complexity:** $O(2N)$
  Each node may be visited twice while adjusting links.

- **Space Complexity:** $O(1)$
  No recursion or stack used.